

Computer Vision – Lecture 08

Deep Learning for Computer Vision

12.05.2025

Bastian Leibe

Visual Computing Institute

RWTH Aachen University

<http://www.vision.rwth-aachen.de/>

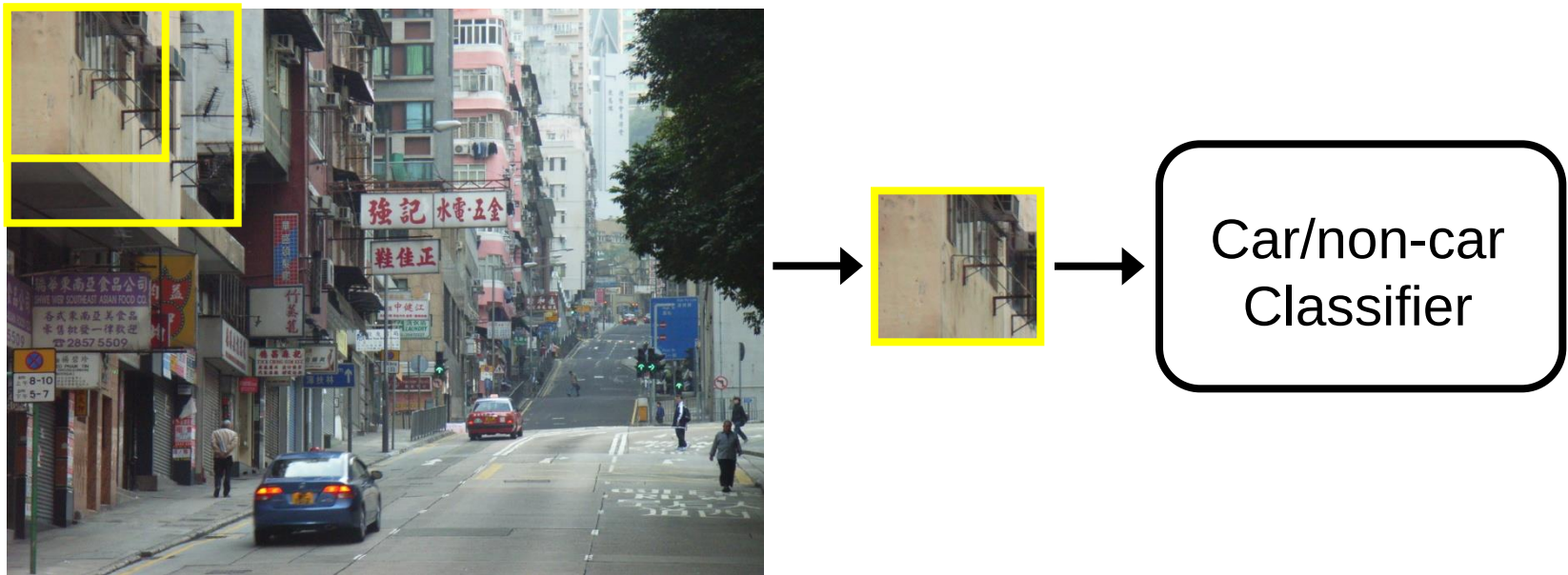
leibe@vision.rwth-aachen.de

Course Outline

- Image Processing Basics
- Segmentation & Grouping
- Object Recognition & Categorization
 - Sliding Window based Object Detection
 - Object Detection case studies
- Deep Learning for Vision
 - Convolutional Neural Networks (CNNs)
 - Deep Learning Background
 - CNNs for Object Detection
 - CNNs for Semantic Segmentation
 - CNNs for Matching
 - Transformer Models
- 3D Reconstruction

Recap: Sliding-Window Object Detection

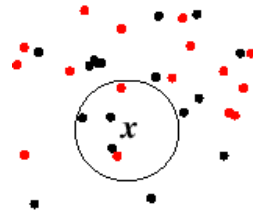
- If object may be in a cluttered scene, slide a window around looking for it.



- Essentially, this is a brute-force approach with many local decisions.

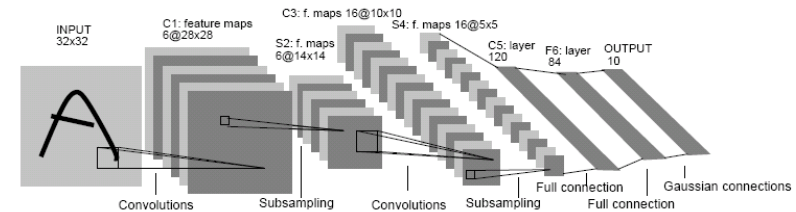
Classifier Construction: Many Choices...

Nearest Neighbor



Berg, Berg, Malik 2005,
Chum, Zisserman 2007,
Boiman, Shechtman, Irani 2008, ...

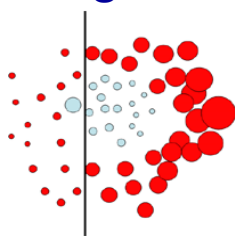
Neural networks



LeCun, Bottou, Bengio, Haffner 1998
Rowley, Baluja, Kanade 1998

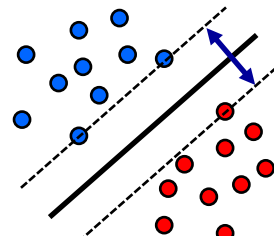
...

Boosting



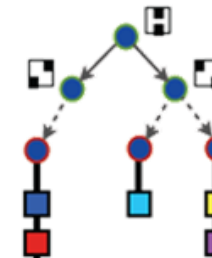
Viola, Jones 2001,
Torralba et al. 2004,
Opelt et al. 2006,
Benenson 2012, ...

Support Vector Machines



Vapnik, Schölkopf 1995,
Papageorgiou, Poggio '01,
Dalal, Triggs 2005,
Vedaldi, Zisserman 2012

Randomized Forests



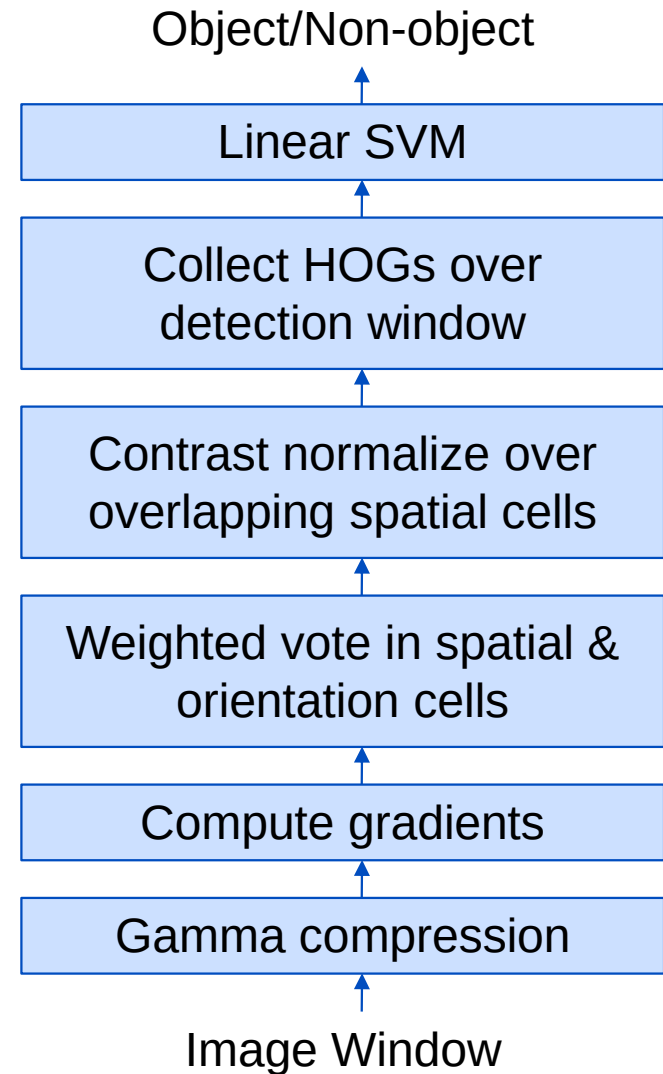
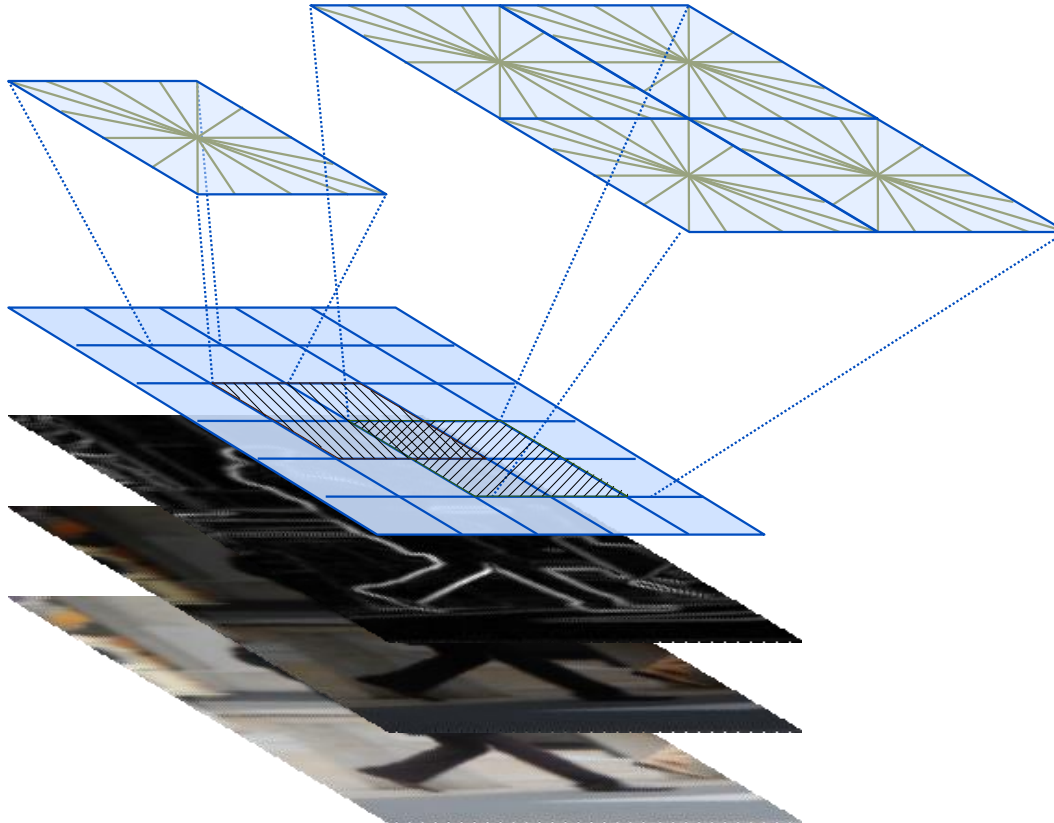
Amit, Geman 1997,
Breiman 2001,
Lepetit, Fua 2006,
Gall, Lempitsky 2009,...

Topics of This Lecture

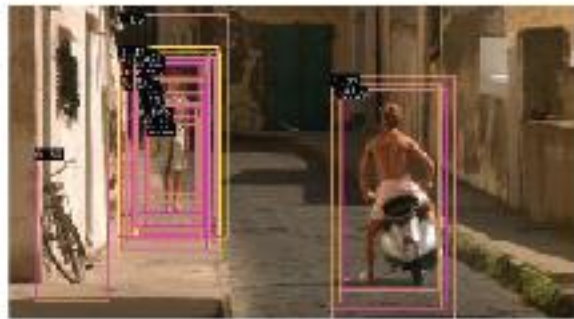
- **Recap: Object Detection with SVM Classifiers**
 - Support Vector Machines
 - HOG Detector
 - DPM Detector
- **Improved Search Strategies**
 - Cascading
 - Region Proposal Generation
- **Deep Learning**
 - Motivation
 - Neural Networks
- **Convolutional Neural Networks**
 - Convolutional Layers
 - Pooling Layers
 - Nonlinearities

Recap: HOG Descriptor Processing Chain

- SVM Classification
 - Typically using a linear SVM



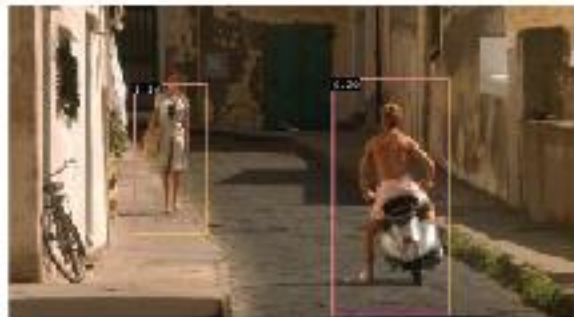
Recap: Non-Maximum Suppression



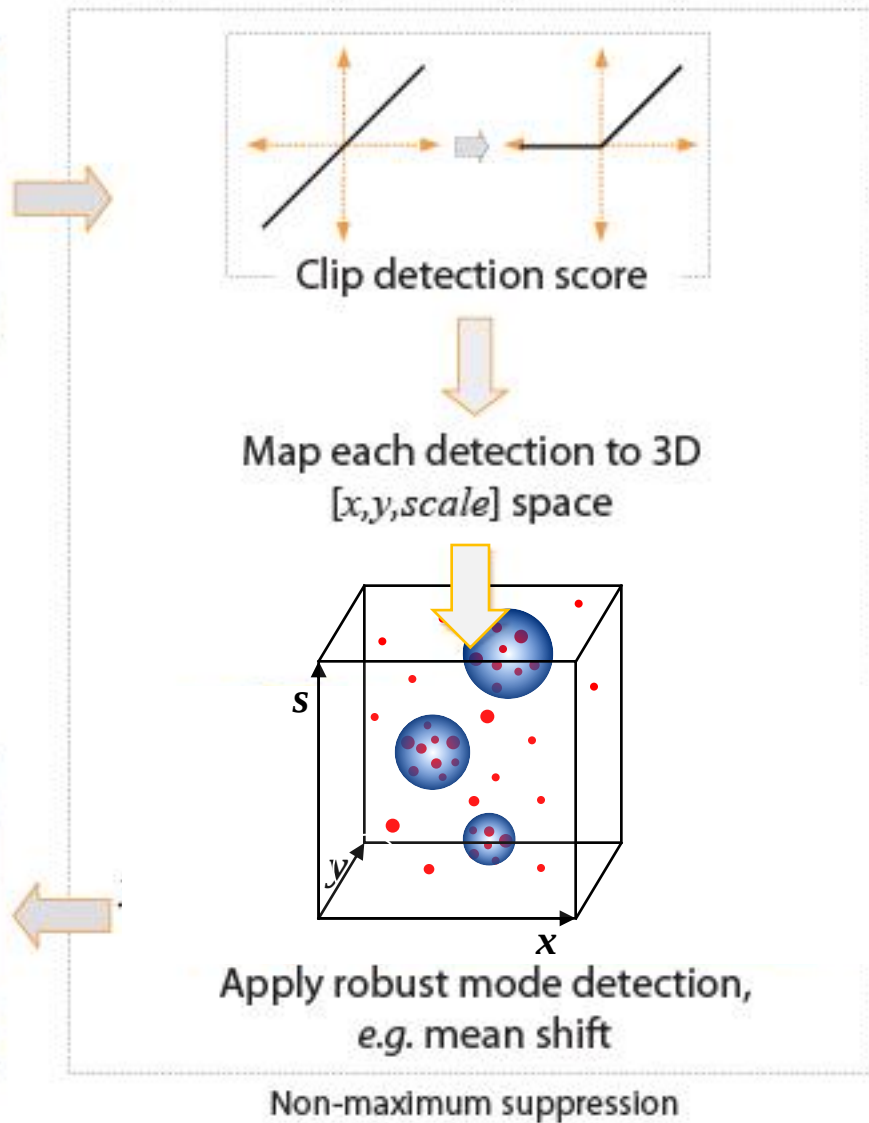
After multi-scale dense scan



Goal



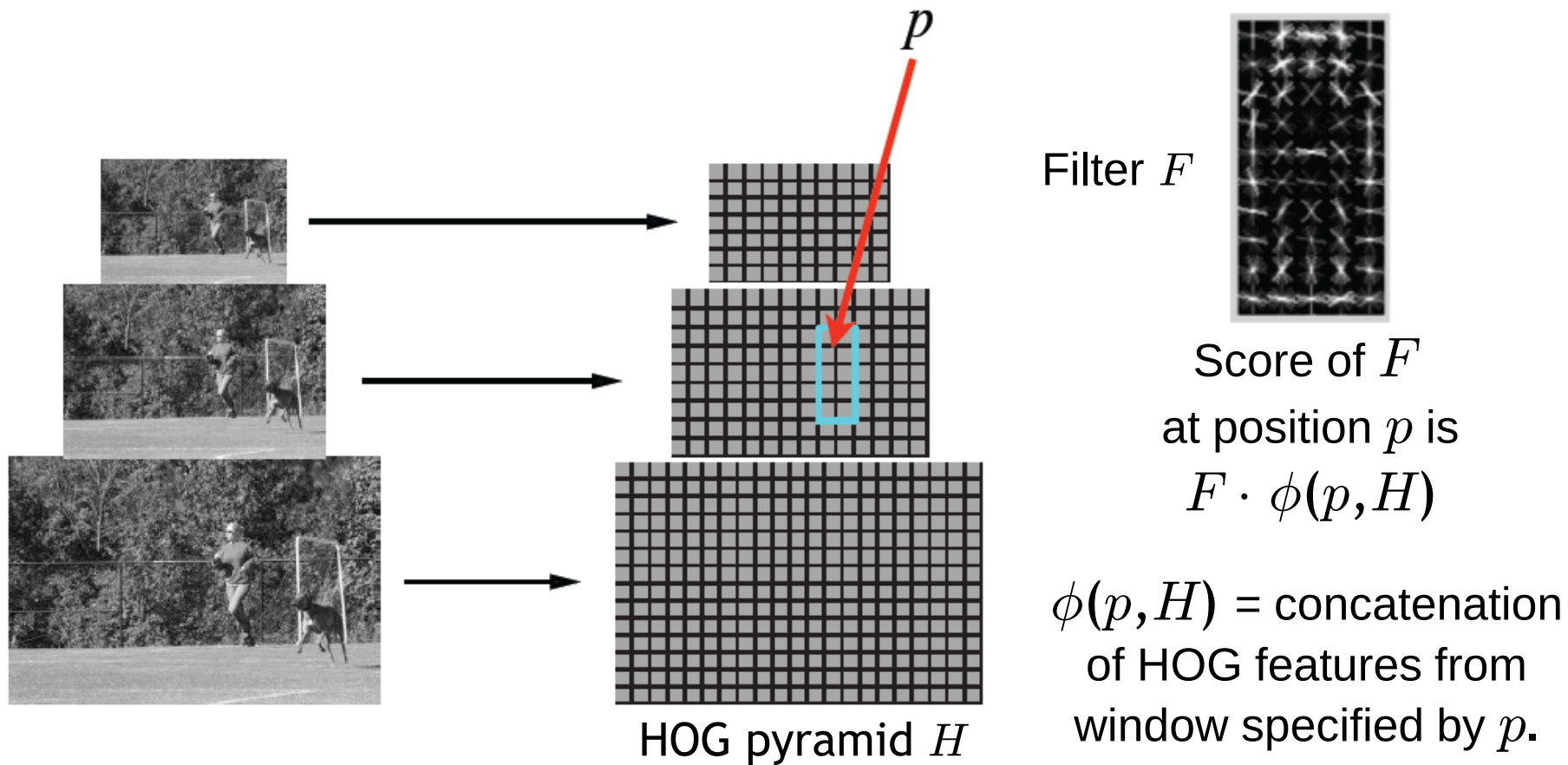
Fusion of multiple detections



Topics of This Lecture

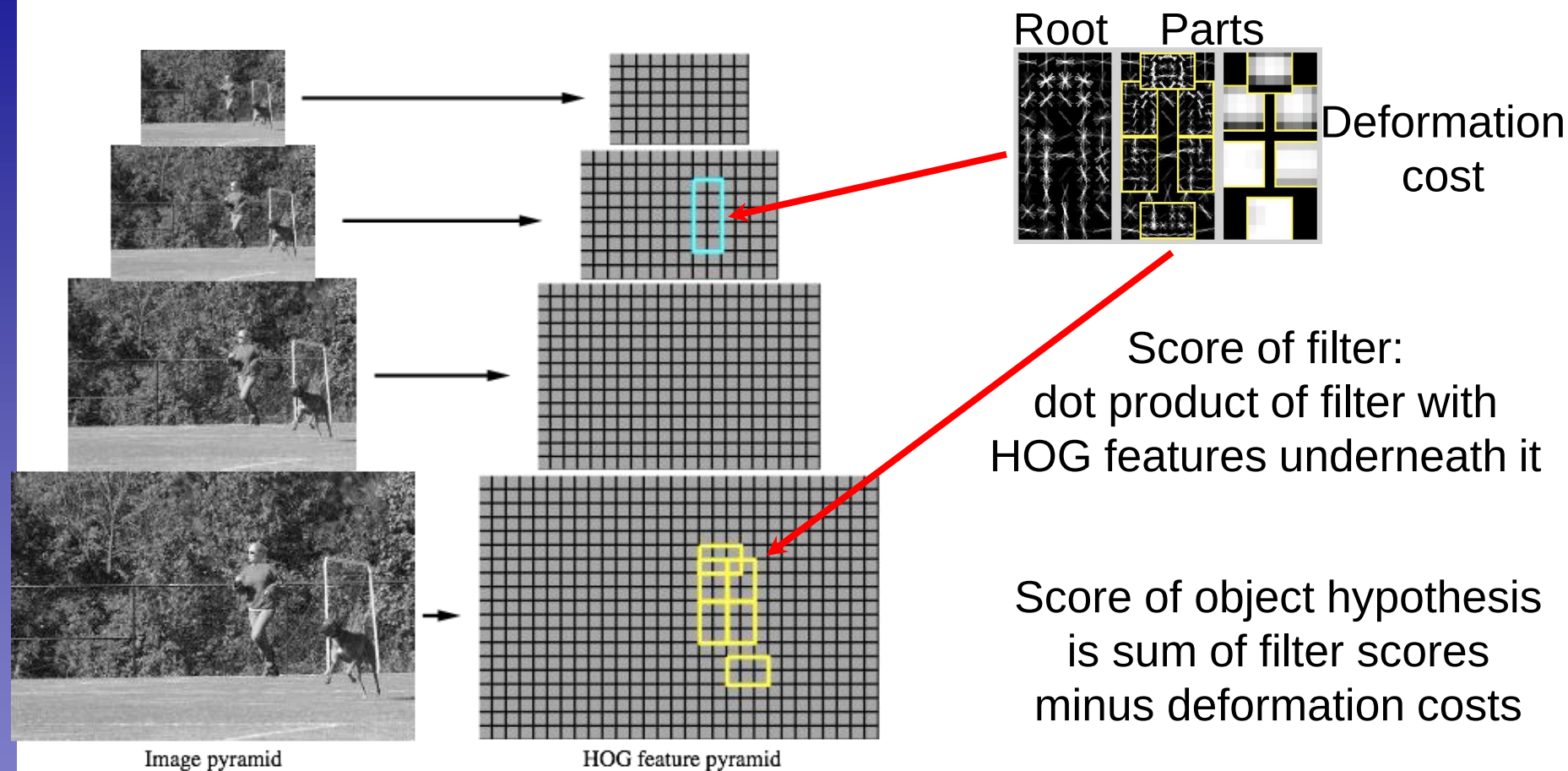
- **Recap: Object Detection with SVM Classifiers**
 - Support Vector Machines
 - HOG Detector
 - **DPM Detector**
- Improved Search Strategies
 - Cascading
 - Region Proposal Generation
- Deep Learning
 - Motivation
 - Neural Networks
- Convolutional Neural Networks
 - Convolutional Layers
 - Pooling Layers
 - Nonlinearities

HOG Sliding-Window Detector



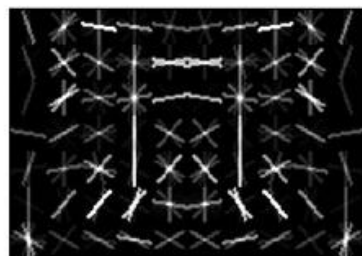
- Array of weights for features in window of HOG pyramid
- Score is dot product of filter and vector

Deformable Part-based Model (DPM)

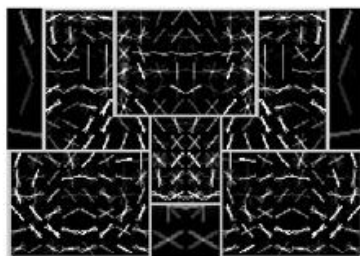


- Multiscale model captures features at two resolutions

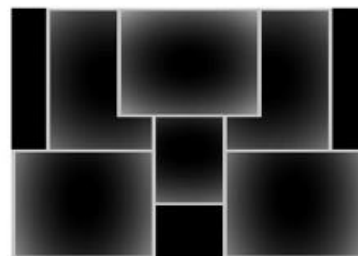
Example: 2-Component Bicycle Model



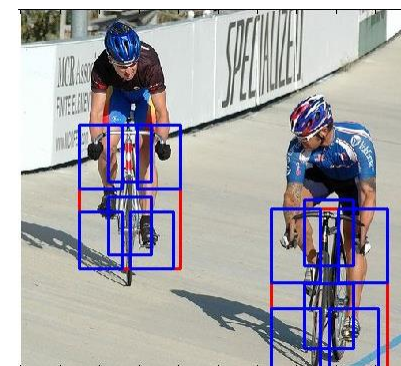
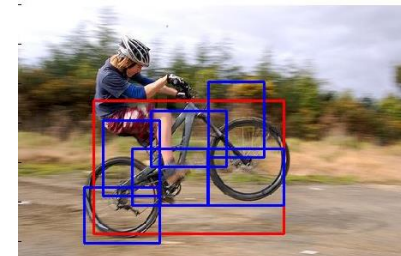
Root filters
coarse resolution



Part filters
finer resolution



Deformation
models



Example
detections

Score of a Hypothesis

$$\text{score}(p_0, \dots, p_n) = \sum_{i=0}^n F_i \cdot \phi(H, p_i) - \sum_{i=1}^n d_i \cdot (dx_i^2, dy_i^2)$$

“data term”

↑ filters

“spatial prior”

↑ displacements

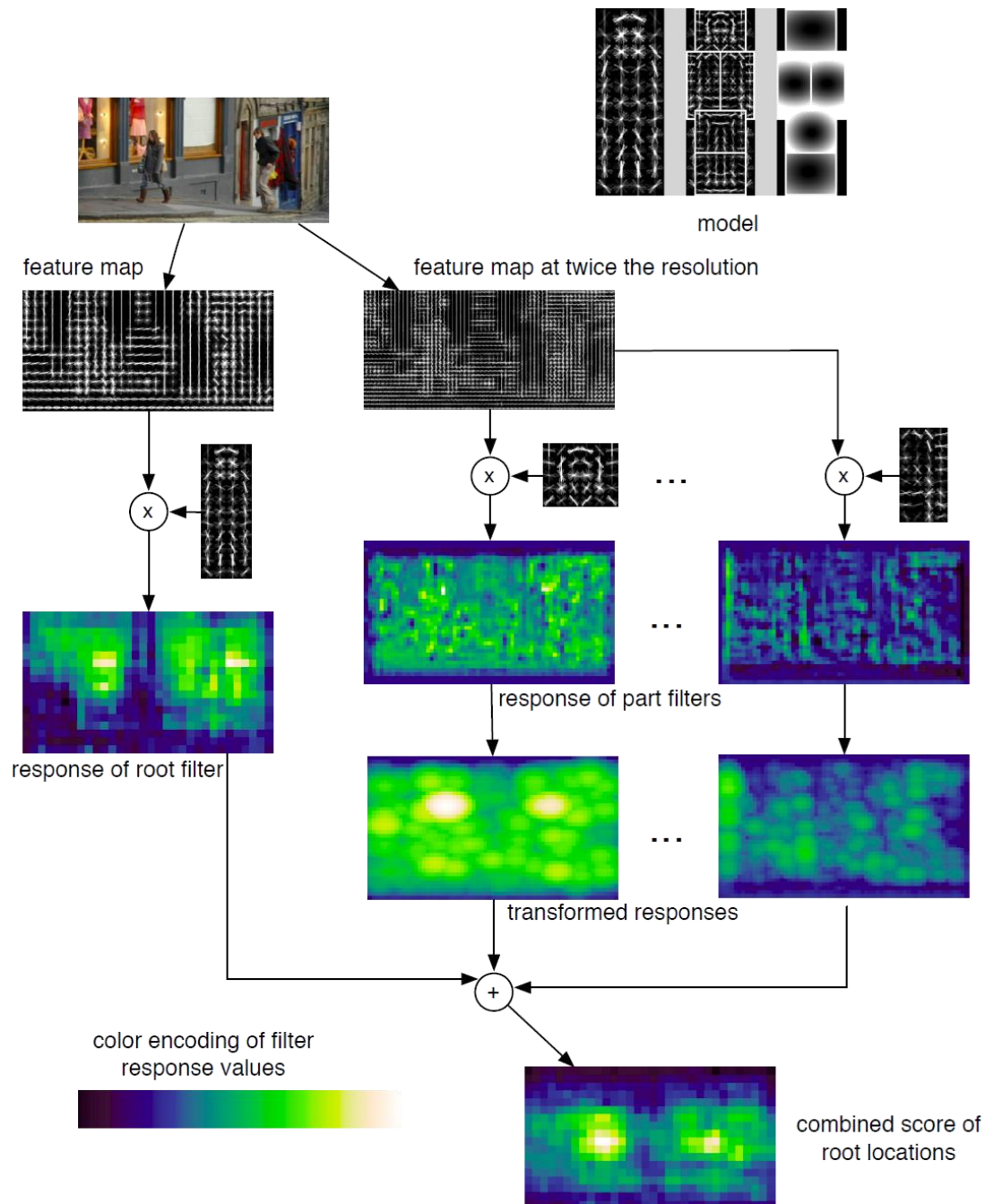
deformation parameters



$$\text{score}(z) = \beta \cdot \Psi(H, z)$$

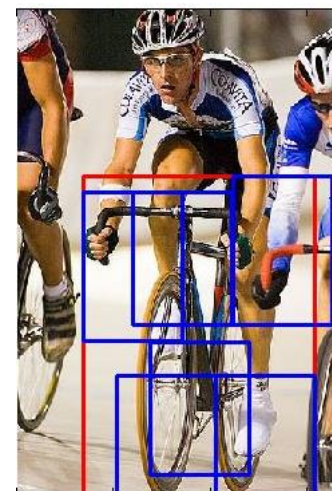
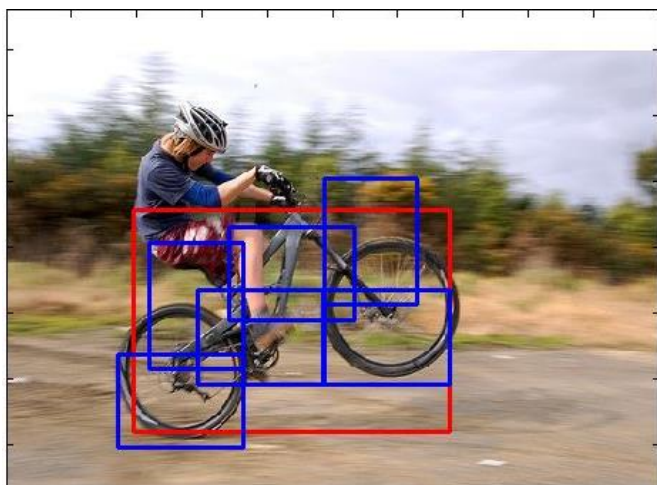
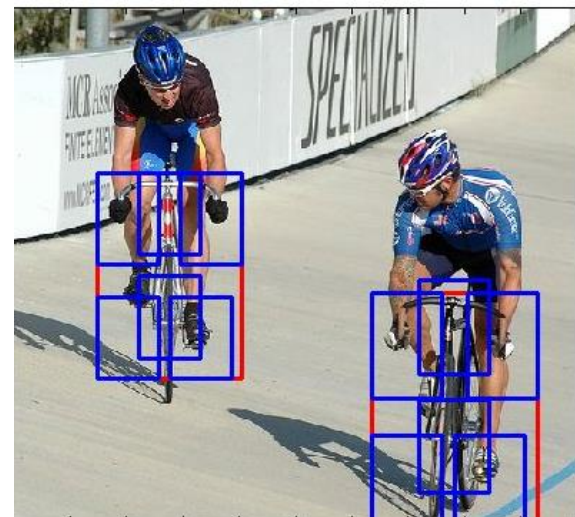
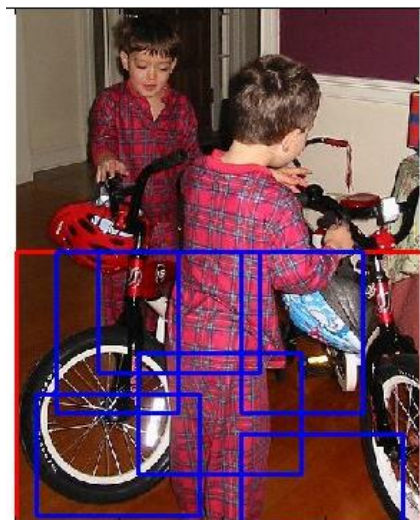
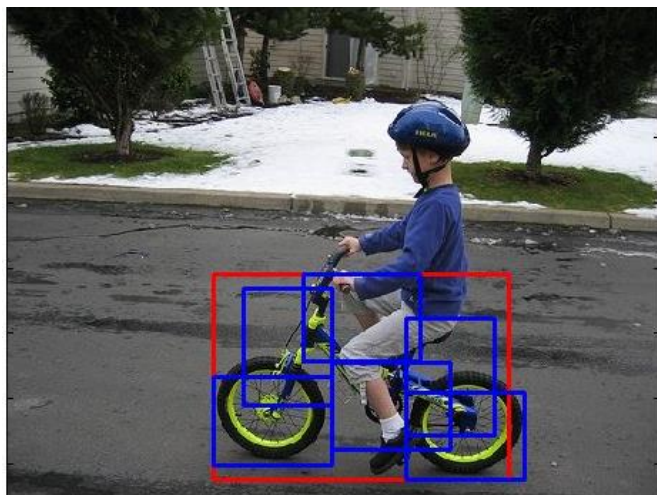
concatenation filters and
deformation parameters

concatenation of HOG
features and part
displacement features



Efficient computation
through Generalized
Distance Transform.

Results: Bicycles

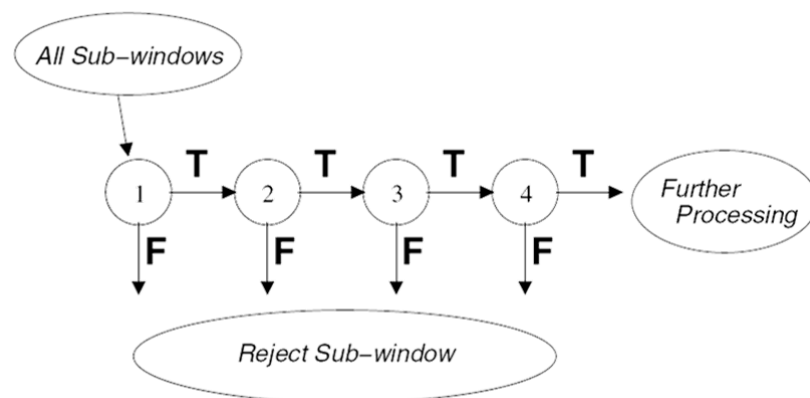


Topics of This Lecture

- Recap: Object Detection with SVM Classifiers
 - Support Vector Machines
 - HOG Detector
 - DPM Detector
- Improved Search Strategies
 - Cascading
 - Region Proposal Generation
- Deep Learning
 - Motivation
 - Neural Networks
- Convolutional Neural Networks
 - Convolutional Layers
 - Pooling Layers
 - Nonlinearities

Cascading Classifiers for Detection

- Even if the classifiers are fast, each new image has a lot of possible windows to search.
- For efficiency, apply less accurate but faster classifiers first to immediately discard windows that clearly appear to be negative; e.g.,
 - Filter for promising regions with an initial inexpensive classifier
 - Build a chain of classifiers, choosing cheap ones with low false negative rates early in the chain

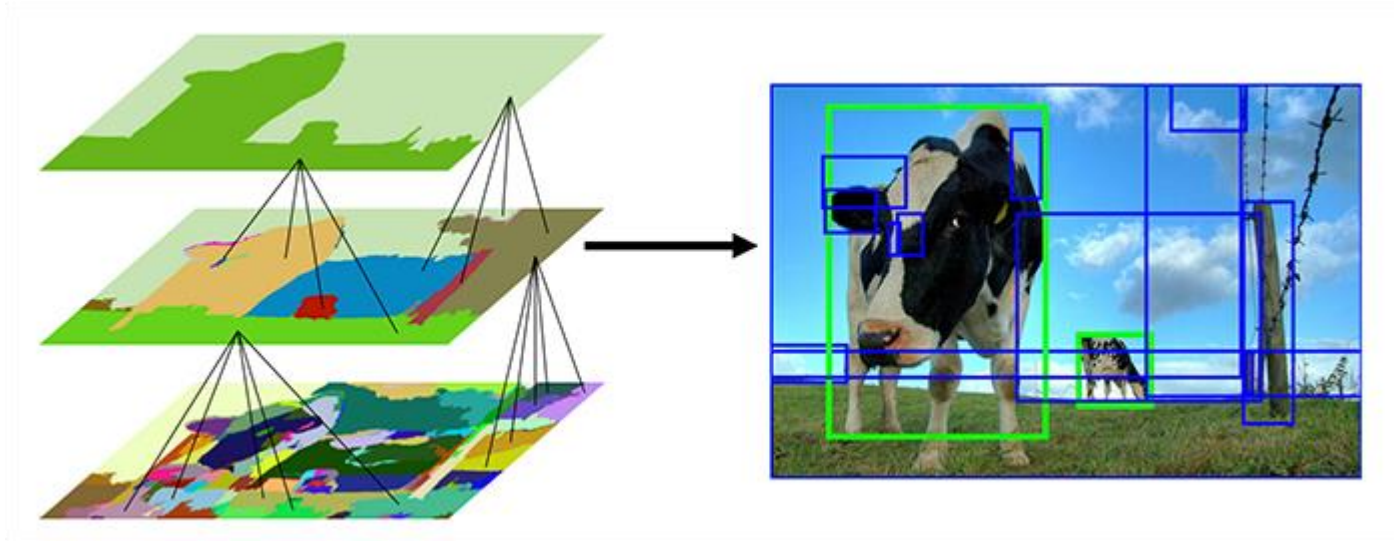


[Fleuret & Geman, IJCV 2001]

[Rowley et al., PAMI 1998]

[Viola & Jones, CVPR 2001]

Extension: Region Proposal Generation



- Alternative to exhaustive sliding-window search
 - E.g., **Selective Search** [Uijlings 2012]
 - Use bottom-up superpixel segmentation
 - Hierarchically group superpixels into larger proposal regions
 - Use those proposal regions to evaluate a classifier
 - Advantage: Focus effort on promising image regions
 - We can spend more computation on more powerful classifiers

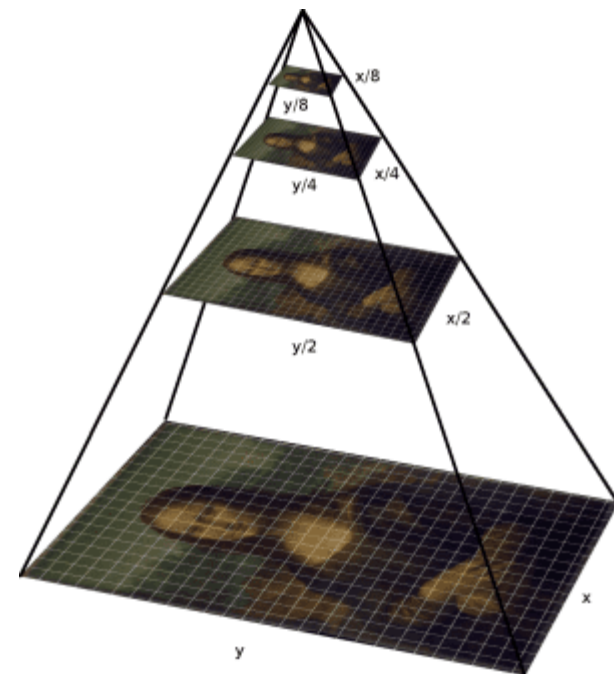
Summary: Sliding-Windows

- Pros

- Simple detection protocol to implement
- Good feature choices critical
- Past successes for certain classes
- Good detectors available

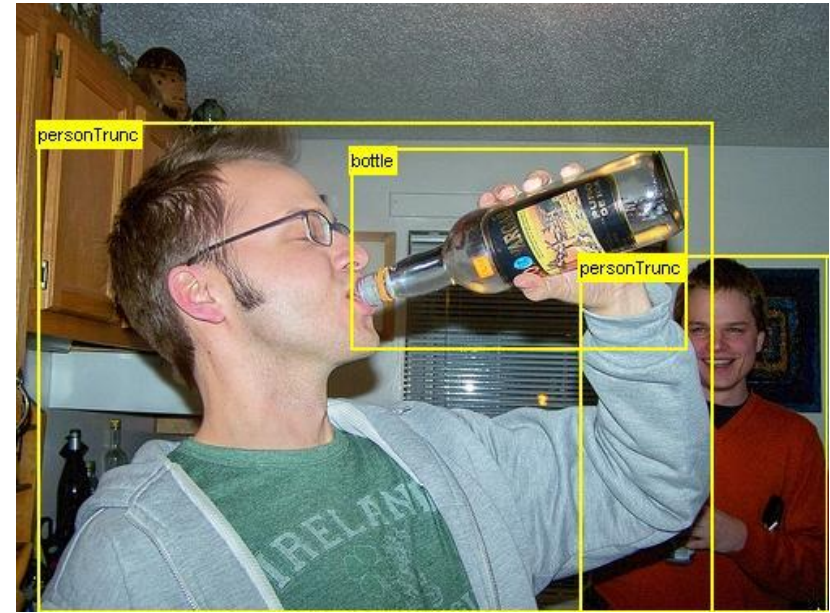
- Cons/Limitations

- High computational complexity
 - For example: 250,000 locations x 30 orientations x 4 scales = 30,000,000 evaluations!
 - This puts tight constraints on the classifiers we can use.
 - If training binary detectors independently, this means cost increases linearly with number of classes.
- With so many windows, false positive rate better be low



Limitations (continued)

- Not all objects are “box” shaped



Limitations (continued)

- If considering windows in isolation, context is lost



Sliding window



Detector's view

Topics of This Lecture

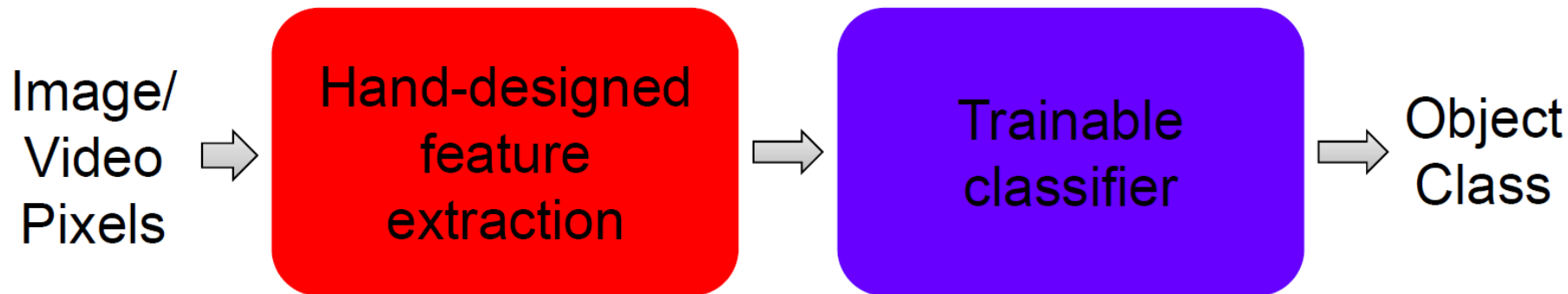
- Recap: Object Detection with SVM Classifiers
 - Support Vector Machines
 - HOG Detector
 - DPM Detector
- Improved Search Strategies
 - Cascading
 - Region Proposal Generation
- Deep Learning
 - Motivation
 - Neural Networks
- Convolutional Neural Networks
 - Convolutional Layers
 - Pooling Layers
 - Nonlinearities

We've finally got there!

A large rectangular image showing an underwater scene. Sunlight rays penetrate the dark blue water from the surface, creating a dramatic, ethereal effect. The rays are concentrated in the center and spread out towards the edges. The water surface is visible at the top, with ripples and reflections of light.

Deep Learning

Traditional Recognition Approach

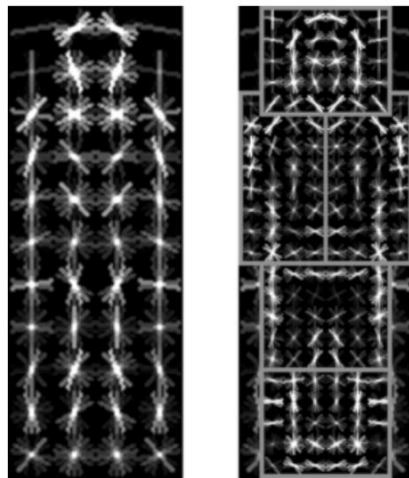


- Characteristics

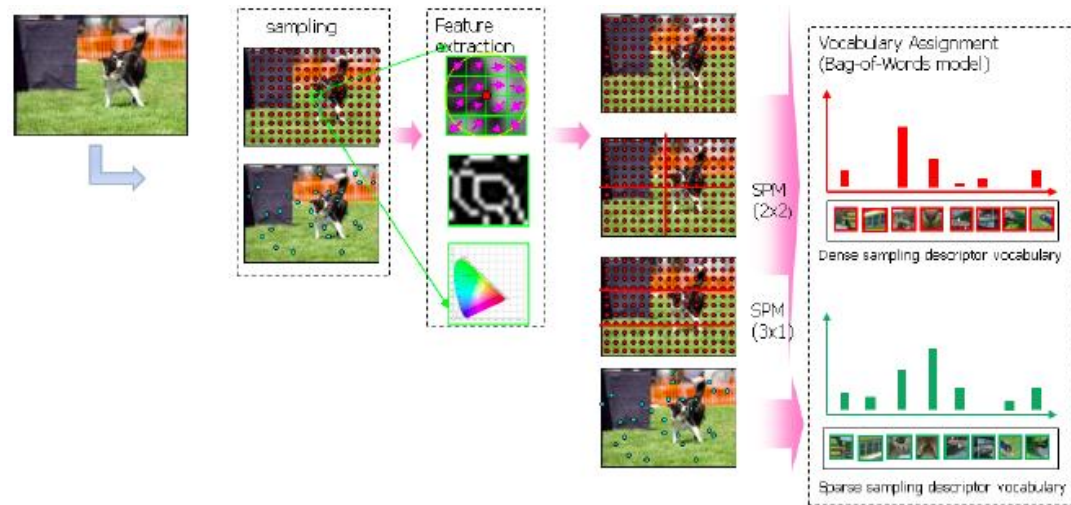
- Features are not learned, but engineered
 - Trainable classifier is often generic (e.g., SVM)
- ⇒ Many successes in 2000-2010.

Traditional Recognition Approach

- Features are key to recent progress in recognition
 - Multitude of hand-designed features currently in use
 - SIFT, HOG,
- ⇒ *Where next? Better classifiers? Or keep building more features?*



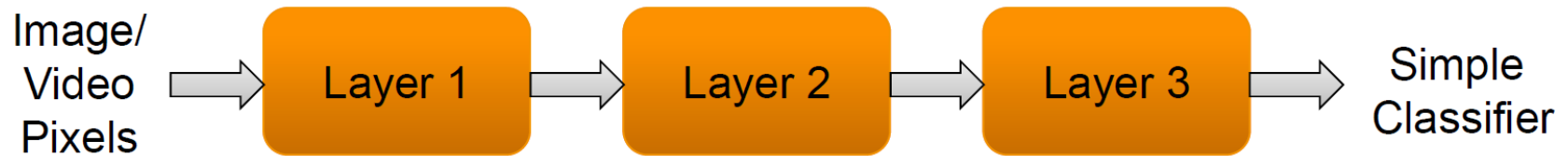
DPM
[Felzenszwalb
et al., PAMI'07]



Dense SIFT+LBP+HOG → BOW → Classifier
[Yan & Huan '10]
(Winner of PASCAL 2010 Challenge)

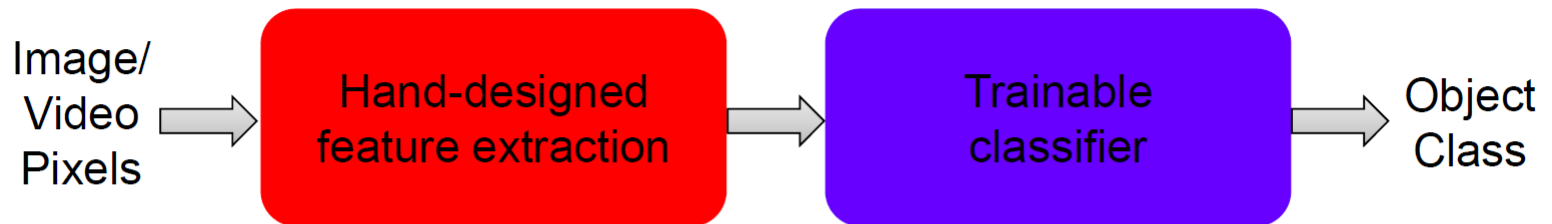
What About Learning the Features?

- Learn a *feature hierarchy* all the way from pixels to classifier
 - Each layer extracts features from the output of previous layer
 - Train all layers jointly



“Shallow” vs. “Deep” Architectures

Traditional recognition: “Shallow” architecture



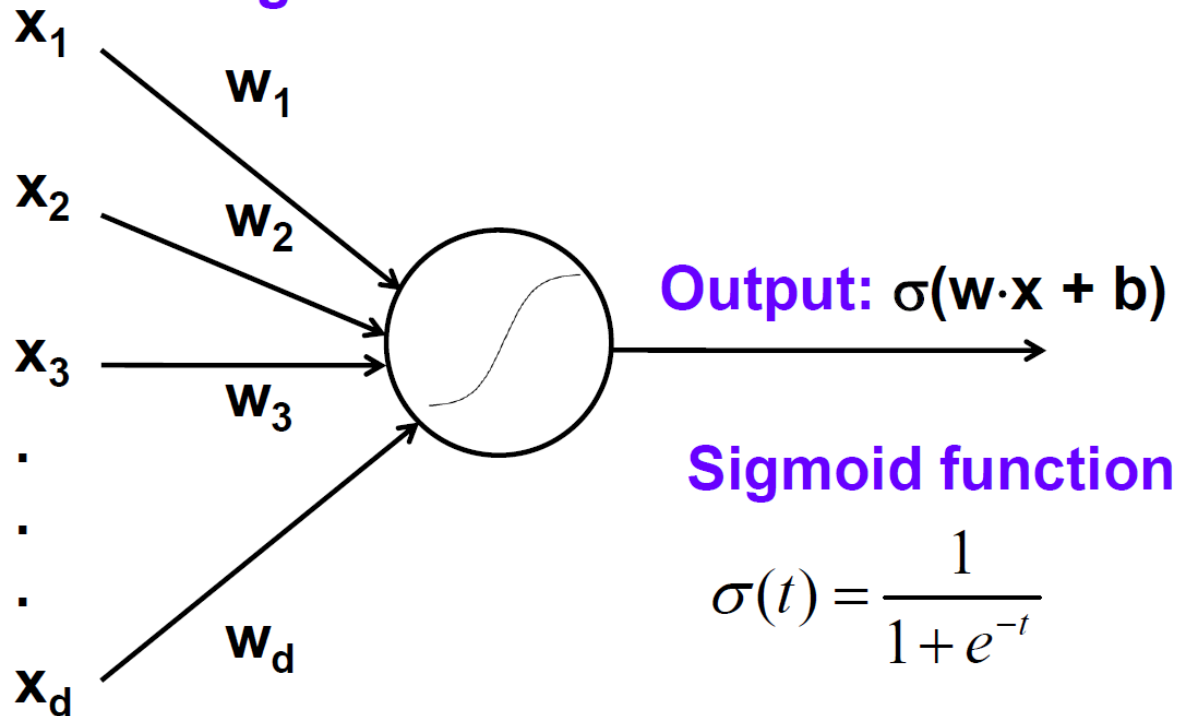
Deep learning: “Deep” architecture



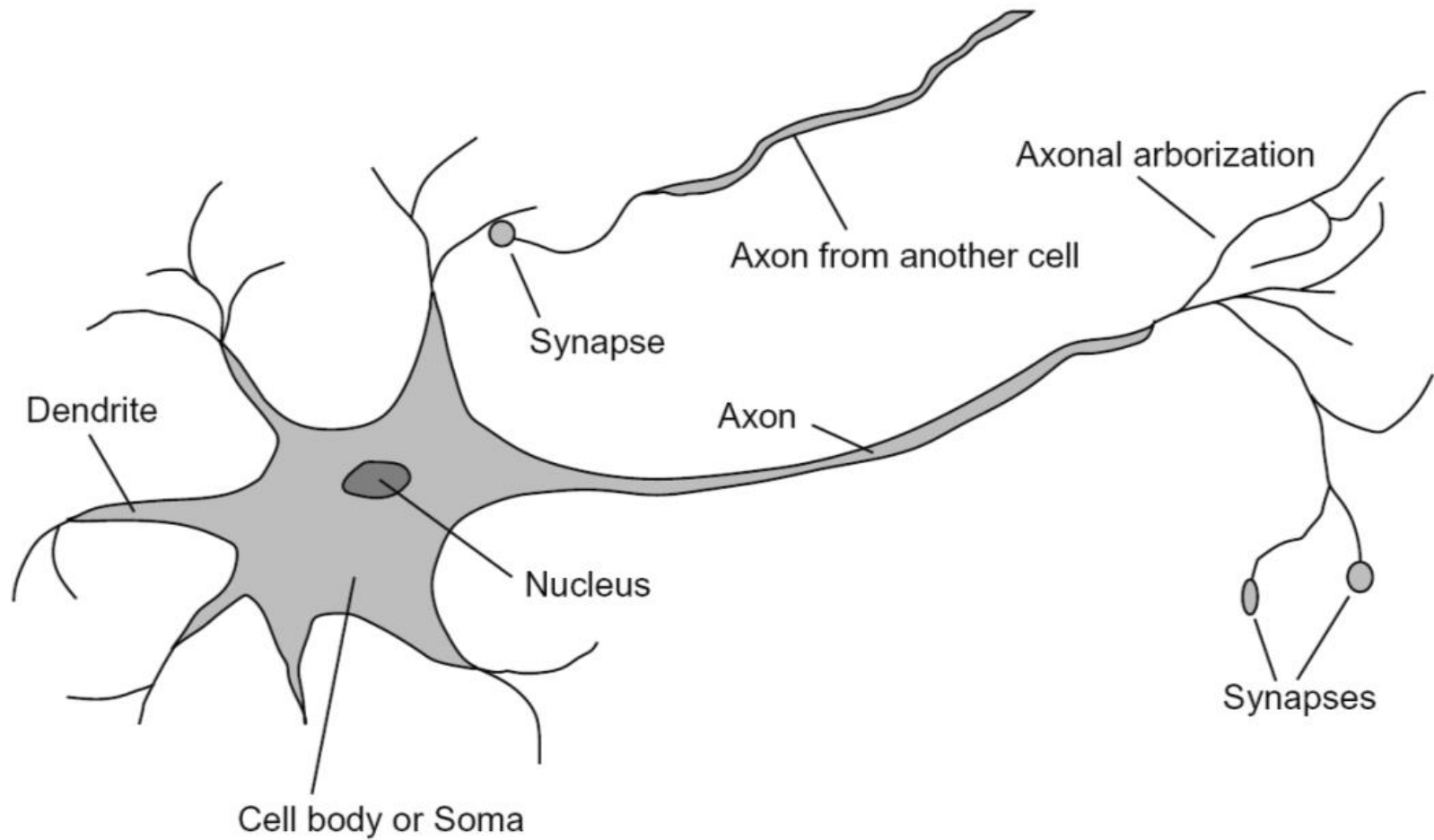
Background: Perceptrons

Input

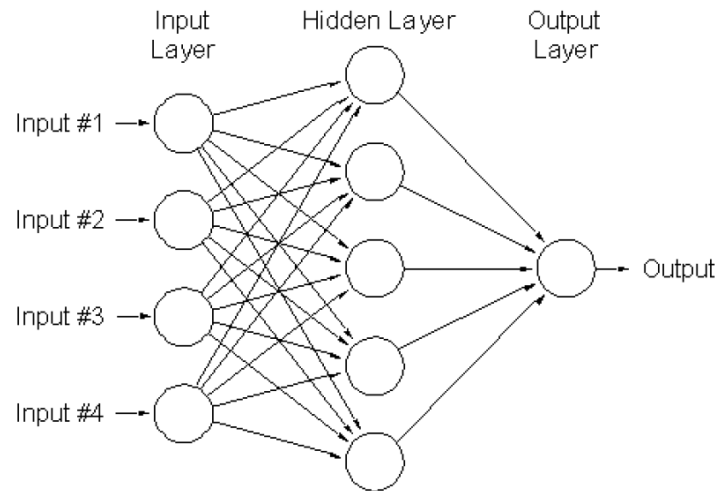
Weights



Inspiration: Neuron Cells



Background: Multi-Layer Neural Networks



- Nonlinear classifier

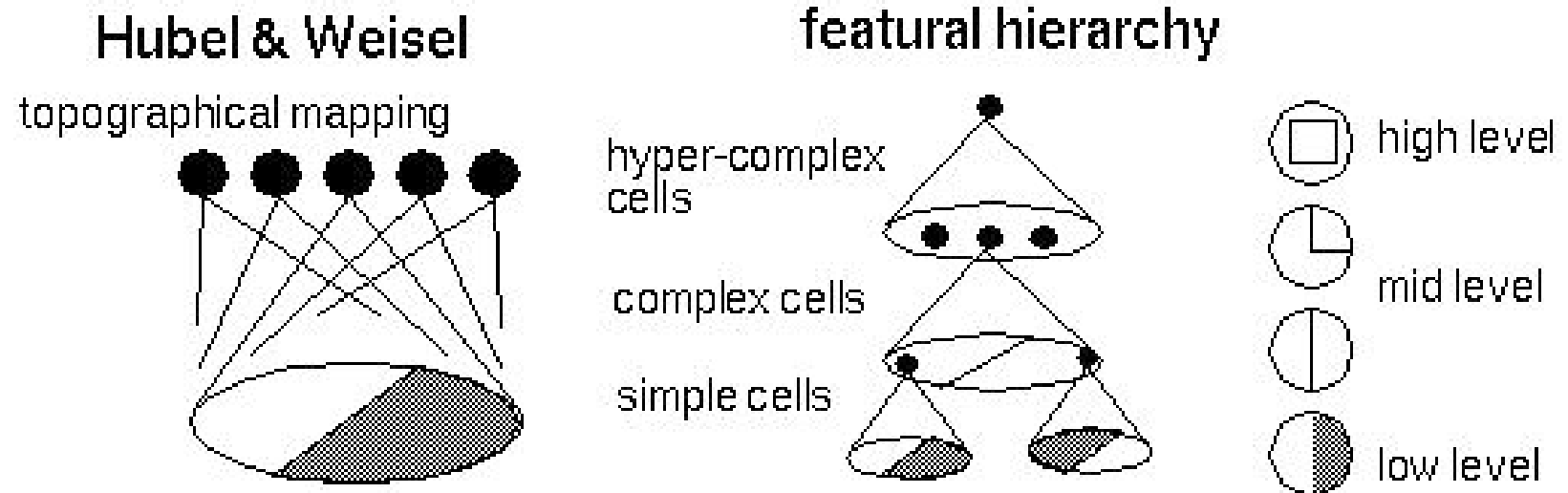
- **Training:** find network weights $\mathbf{w}$ to minimize the error between true training labels t_n and estimated labels $f_{\mathbf{w}}(x_n)$:

$$E(\mathbf{W}) = \sum^n L(t_n, f(\mathbf{x}_n; \mathbf{W}))$$

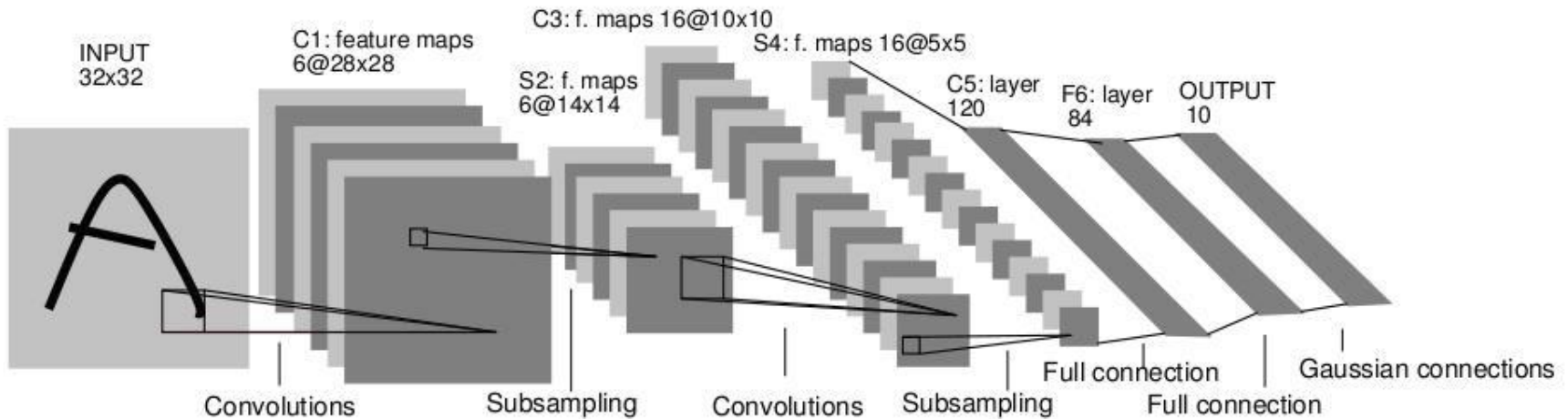
- Minimization can be done by gradient descent, provided f is differentiable
 - Efficient gradient computation with **error backpropagation**.

Hubel/Wiesel Architecture

- D. Hubel, T. Wiesel (1959, 1962, Nobel Prize 1981)
 - Visual cortex consists of a hierarchy of *simple*, *complex*, and *hyper-complex* cells



Convolutional Neural Networks (CNN, ConvNet)



- Neural network with specialized connectivity structure
 - Stack multiple stages of feature extractors
 - Higher stages compute more global, more invariant features
 - Classification layer at the end

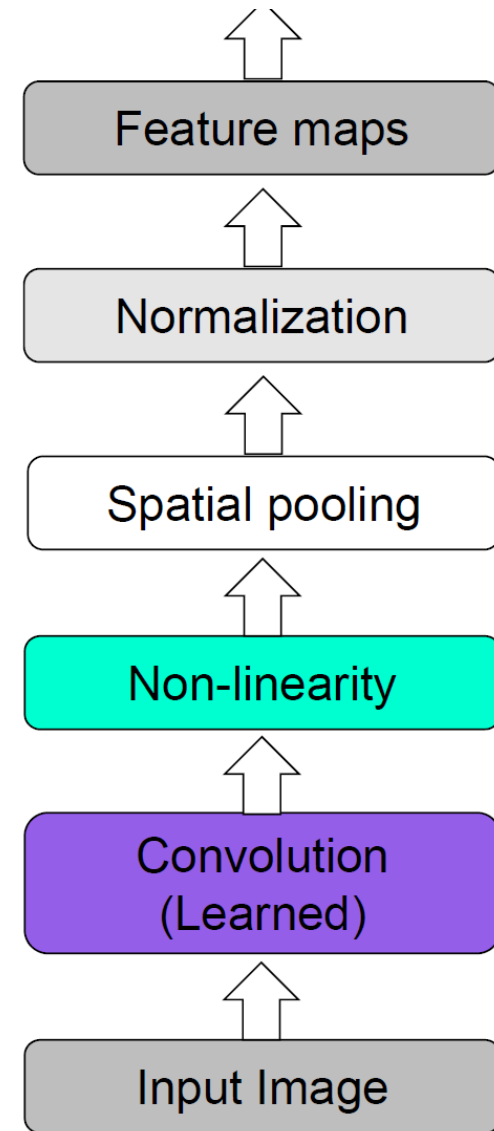
Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, [Gradient-based learning applied to document recognition](#), Proceedings of the IEEE 86(11): 2278–2324, 1998.

Topics of This Lecture

- Recap: Object Detection with SVM Classifiers
 - Support Vector Machines
 - HOG Detector
 - DPM Detector
- Improved Search Strategies
 - Cascading
 - Region Proposal Generation
- Deep Learning
 - Motivation
 - Neural Networks
- Convolutional Neural Networks
 - Convolutional Layers
 - Pooling Layers
 - Nonlinearities

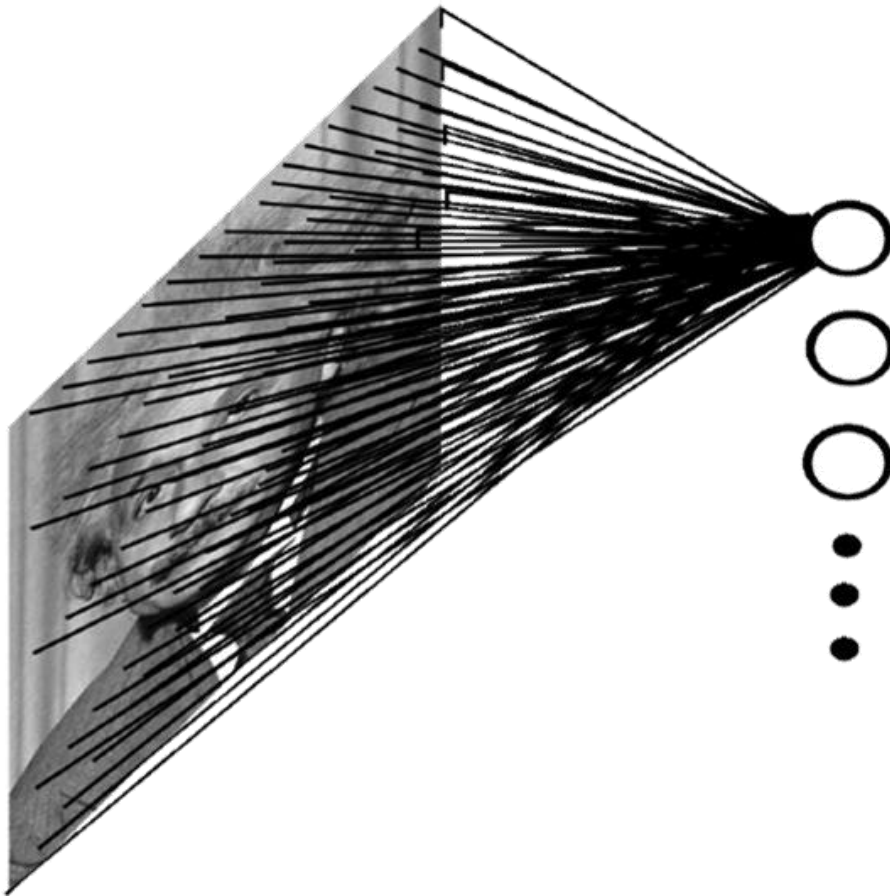
Convolutional Networks: Structure

- Feed-forward feature extraction
 1. Convolve input with learned filters
 2. Non-linearity
 3. Spatial pooling
 4. (Normalization)
- Supervised training of convolutional filters by back-propagating classification error



Convolutional Networks: Intuition

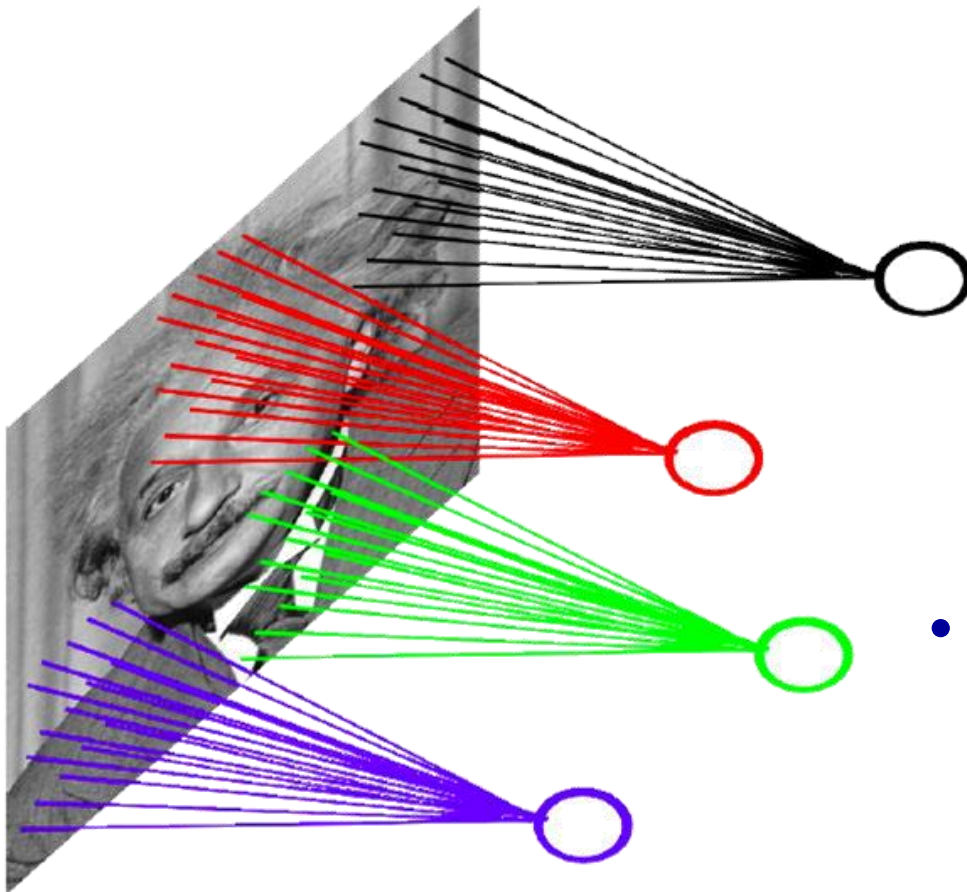
- Fully connected network
 - E.g. 1000×1000 image
1M hidden units
 - ⇒ 1T parameters!



- Ideas to improve this
 - Spatial correlation is local

Convolutional Networks: Intuition

- Locally connected net
 - E.g. 1000×1000 image
1M hidden units
 10×10 receptive fields
 $\Rightarrow$ 100M parameters!

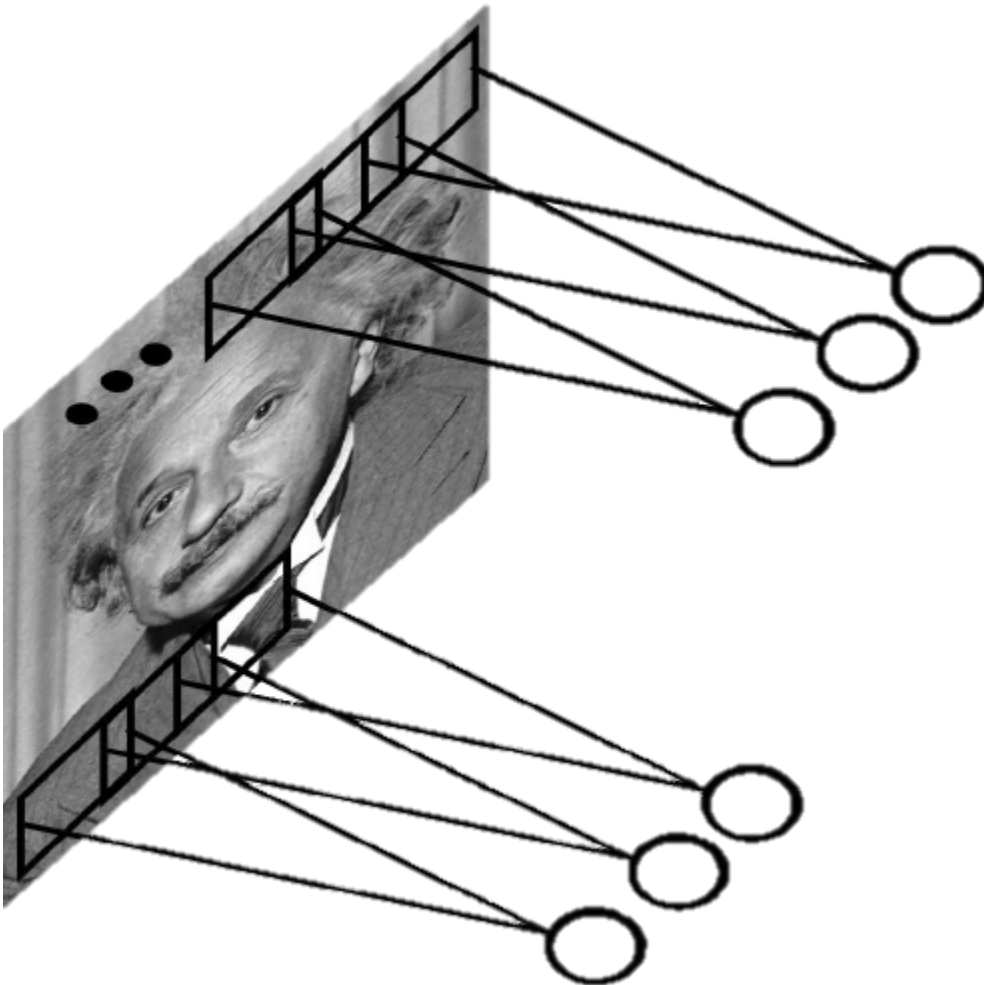


- Ideas to improve this
 - Spatial correlation is local
 - Want translation invariance

Convolutional Networks: Intuition

- Convolutional net

- Share the same parameters across different locations
- Convolutions with learned kernels



Convolutional Networks: Intuition

- Convolutional net

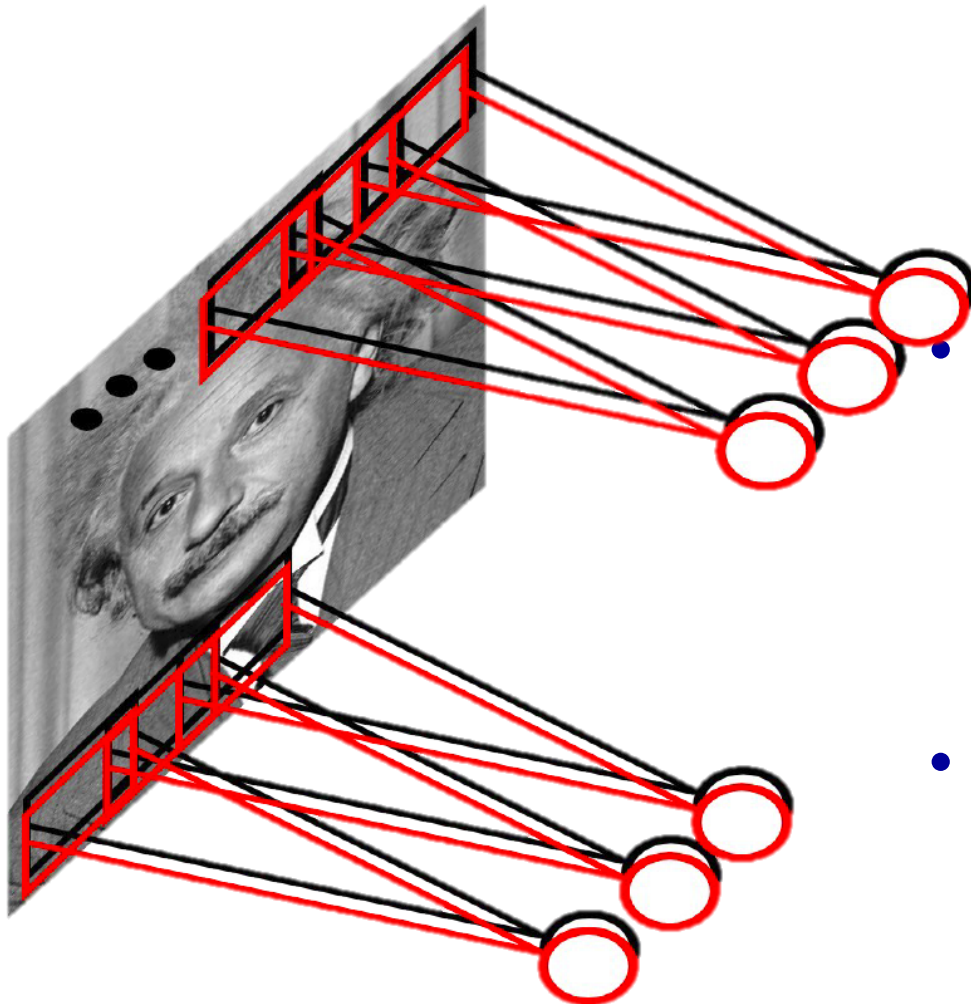
- Share the same parameters across different locations
- Convolutions with learned kernels

- Learn *multiple* filters

- E.g. 1000×1000 image
100 filters
 10×10 filter size
 $\Rightarrow$ 10k parameters

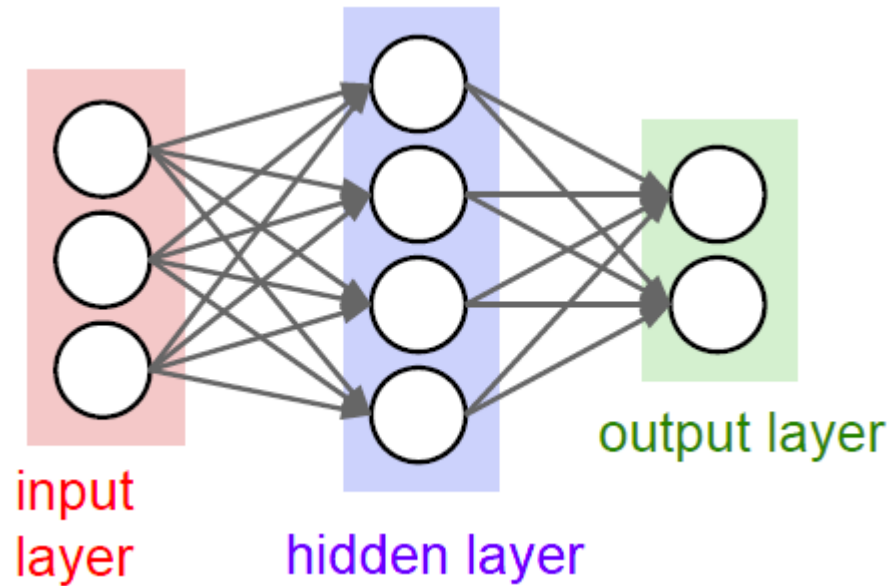
- Result: Response map

- size: $1000 \times 1000 \times 100$
- Only memory, not params!

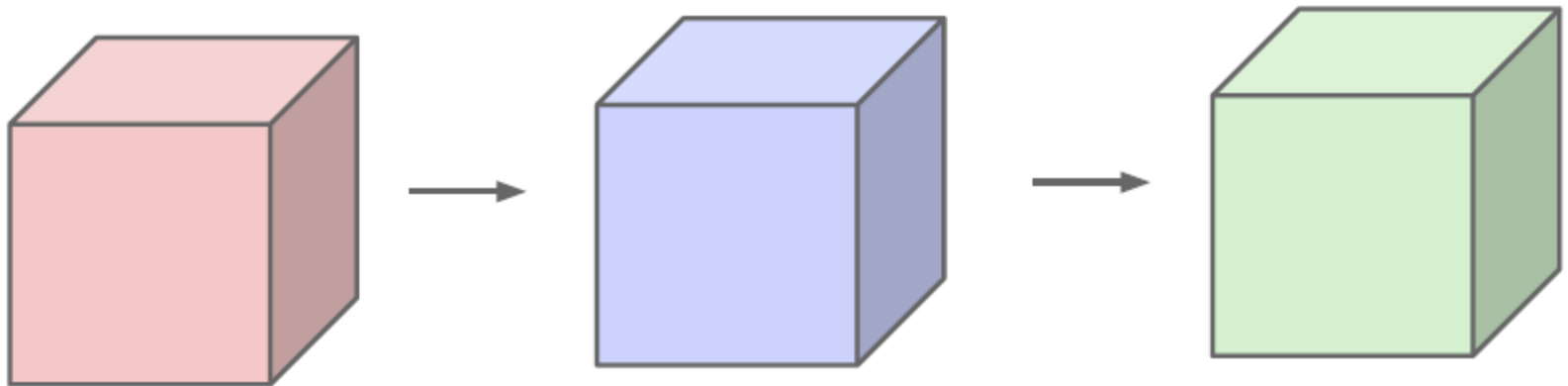


Important Conceptual Shift

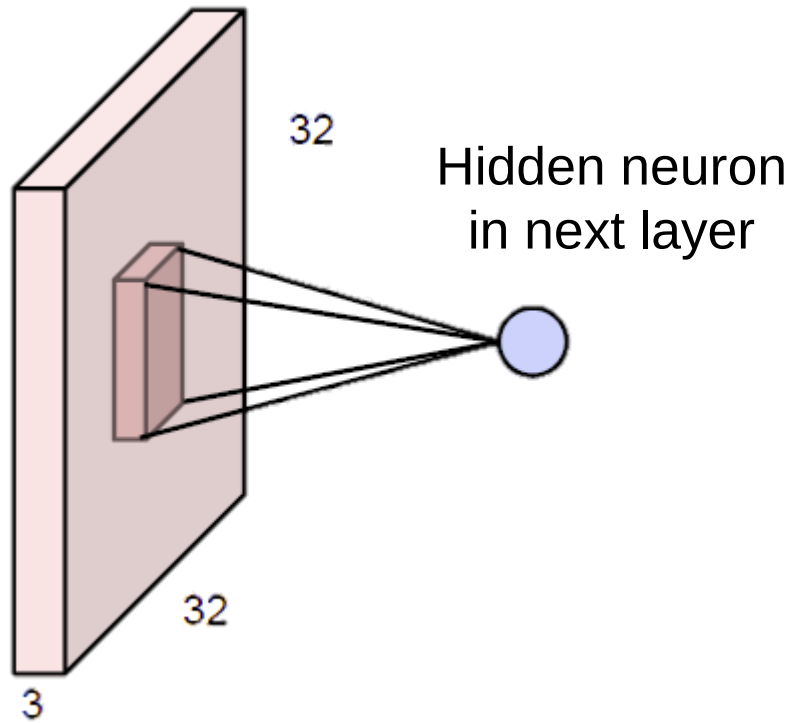
- Before



- Now:



Convolution Layers



Example

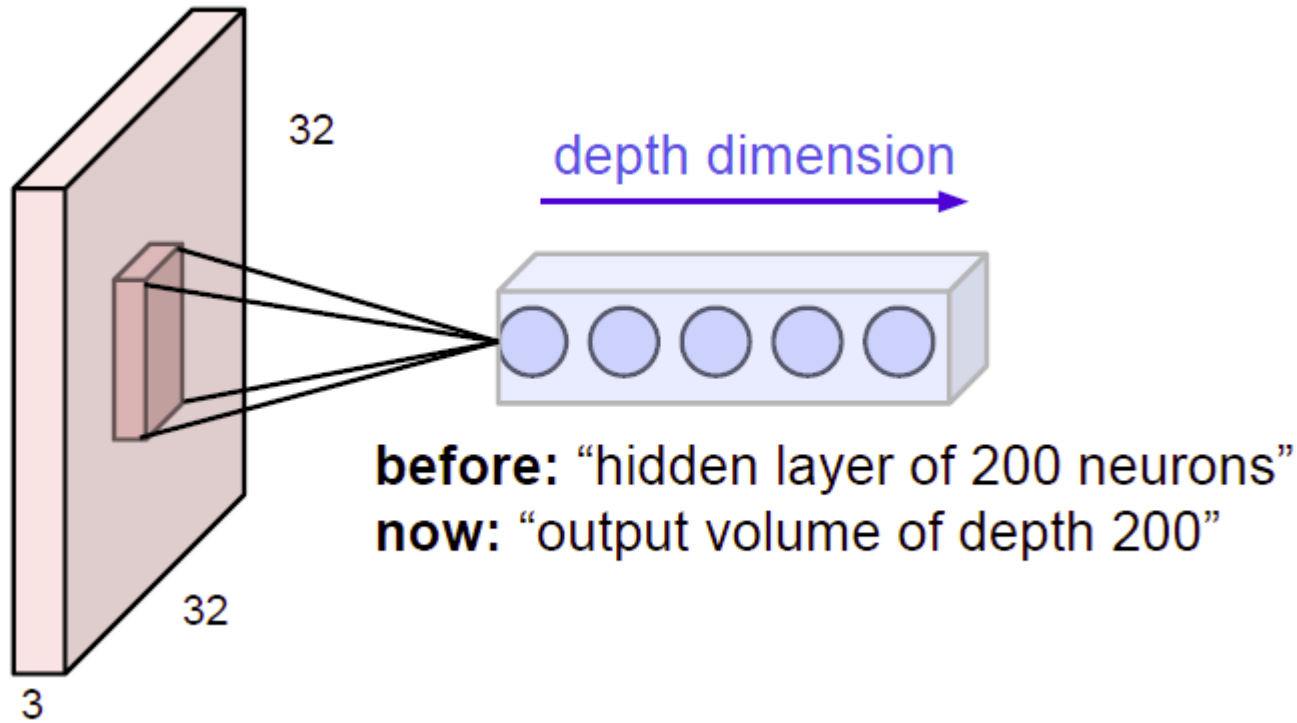
image: $32 \times 32 \times 3$ volume

Before: Full connectivity
 $32 \times 32 \times 3$ weights

Now: Local connectivity
One neuron connects to, e.g.,
 $5 \times 5 \times 3$ region.
 $\Rightarrow$ Only $5 \times 5 \times 3$ **shared weights**.

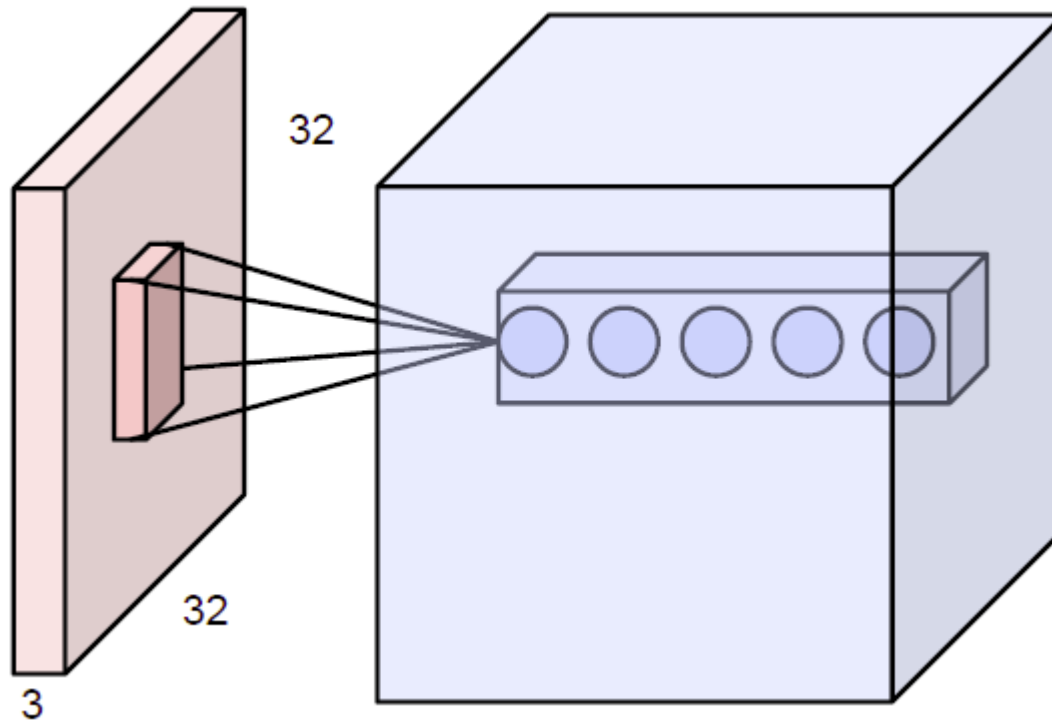
- Note: Connectivity is
 - Local in space (5×5 inside 32×32)
 - But full in depth (all 3 depth channels)

Convolution Layers

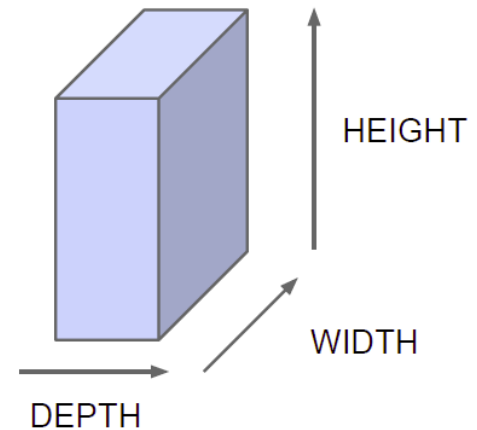


- All Neural Net activations arranged in 3 dimensions
 - Multiple neurons all looking at the same input region, stacked in depth

Convolution Layers

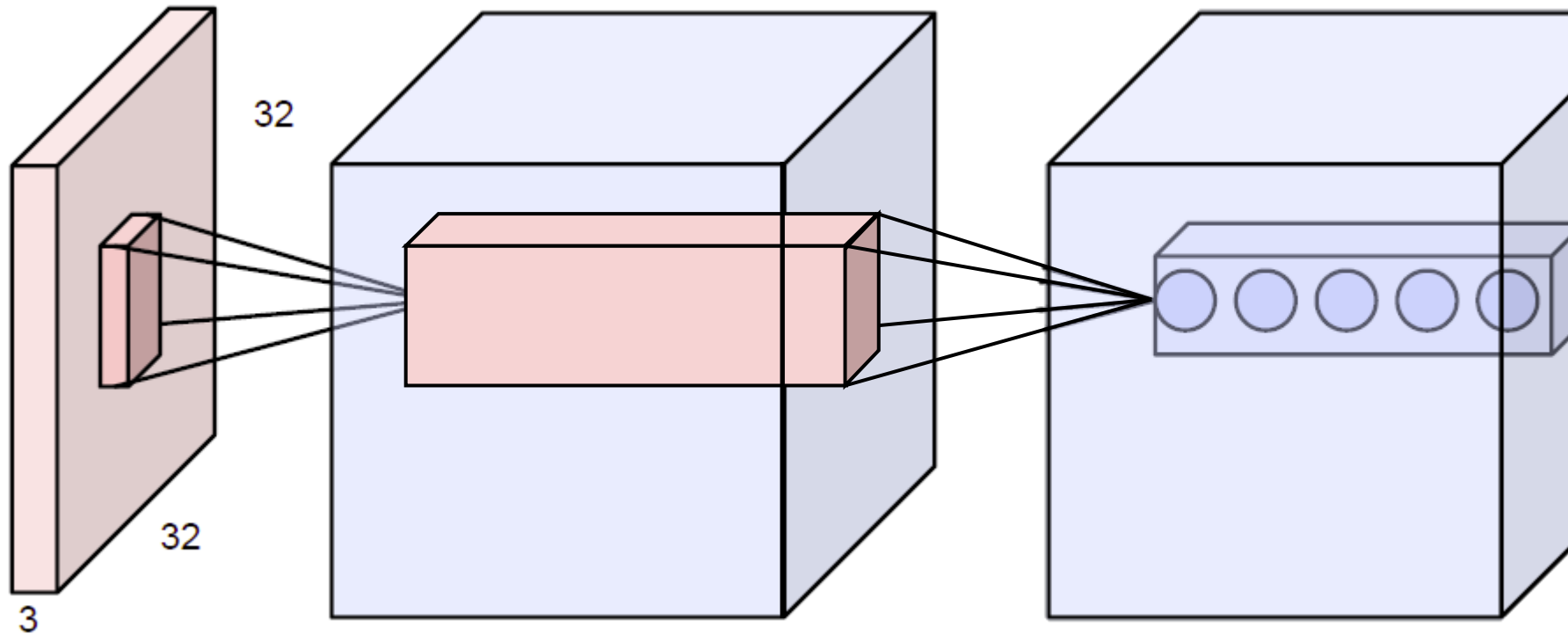


Naming convention:



- All Neural Net activations arranged in 3 dimensions
 - Multiple neurons all looking at the same input region, stacked in depth
 - Form a single $[1 \times 1 \times \text{depth}]$ depth column in output volume.

Convolution Layers



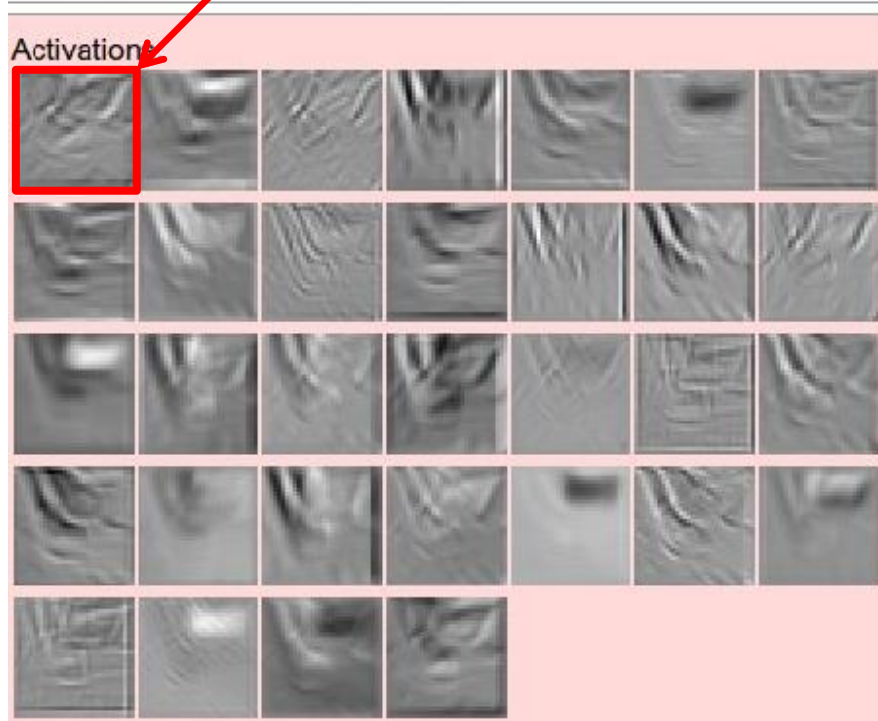
- All Neural Net activations arranged in 3 dimensions
 - Convolution layers can be stacked
 - The filters of the next layer then operate on the full activation volume.
 - Filters are local in (x,y), but densely connected in depth.

Activation Maps of Convolutional Filters

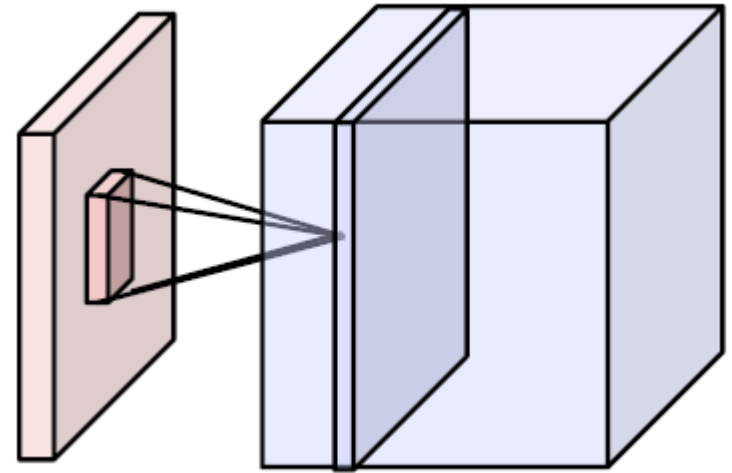
Activations:



5×5 filters

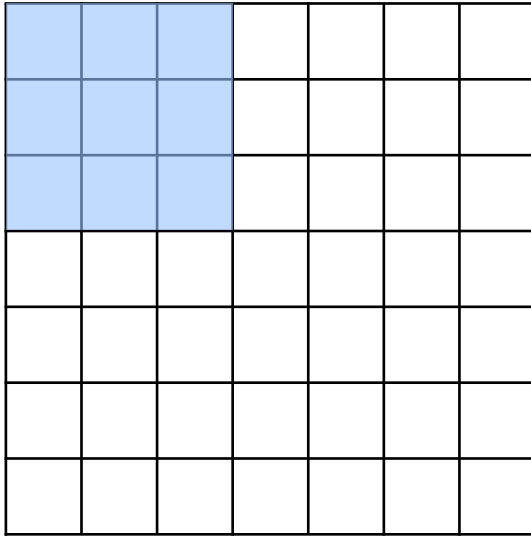


Activation maps



Each activation map is a depth slice through the output volume.

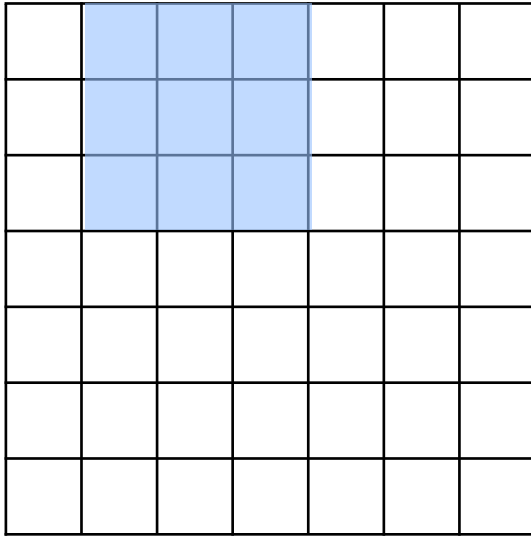
Convolution Layers



Example:
 7×7 input
assume 3×3 connectivity
stride 1

- Replicate this column of hidden neurons across space, with some **stride**.

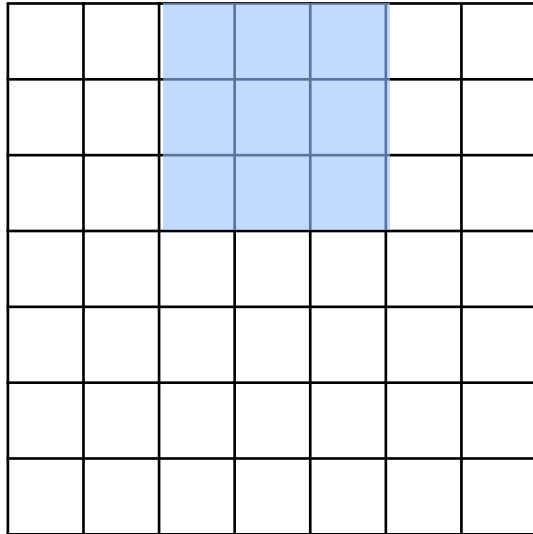
Convolution Layers



Example:
 7×7 input
assume 3×3 connectivity
stride 1

- Replicate this column of hidden neurons across space, with some **stride**.

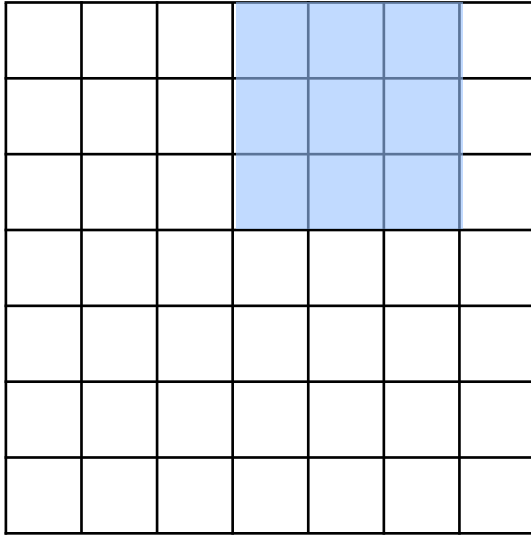
Convolution Layers



Example:
 7×7 input
assume 3×3 connectivity
stride 1

- Replicate this column of hidden neurons across space, with some **stride**.

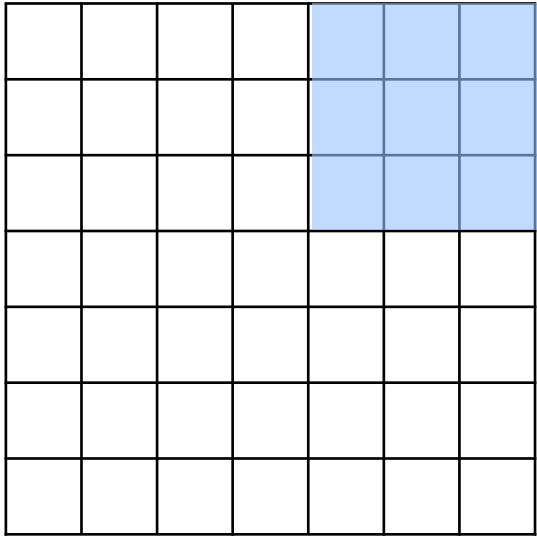
Convolution Layers



Example:
 7×7 input
assume 3×3 connectivity
stride 1

- Replicate this column of hidden neurons across space, with some **stride**.

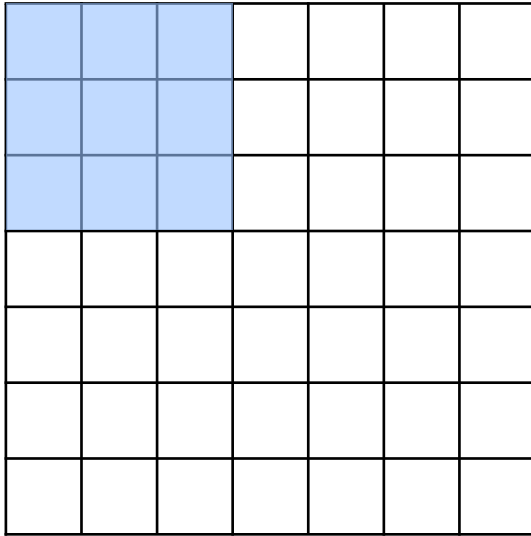
Convolution Layers



Example:
 7×7 input
assume 3×3 connectivity
stride 1
 $\Rightarrow 5 \times 5$ output

- Replicate this column of hidden neurons across space, with some **stride**.

Convolution Layers



Example:

7×7 input

assume 3×3 connectivity

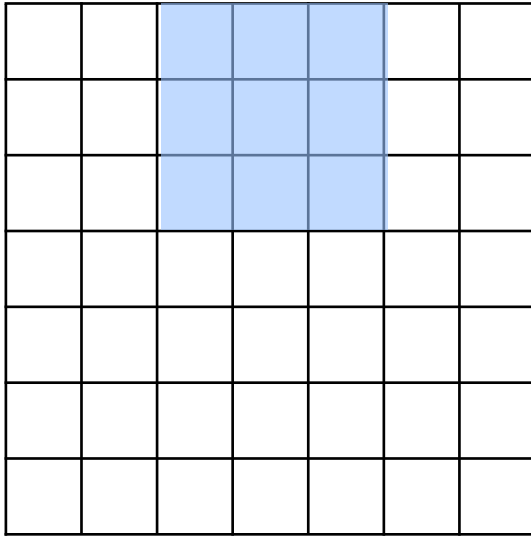
stride 1

$\Rightarrow 5 \times 5$ output

What about stride 2?

- Replicate this column of hidden neurons across space, with some **stride**.

Convolution Layers



Example:

7×7 input

assume 3×3 connectivity

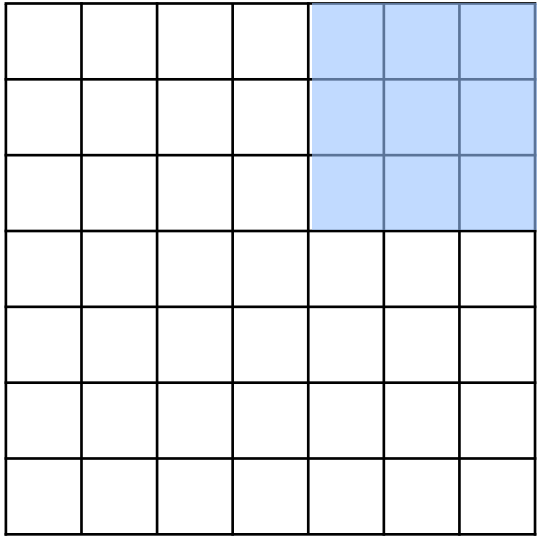
stride 1

$\Rightarrow 5 \times 5$ output

What about stride 2?

- Replicate this column of hidden neurons across space, with some **stride**.

Convolution Layers



Example:

7×7 input

assume 3×3 connectivity

stride 1

$\Rightarrow 5 \times 5$ output

What about stride 2?

$\Rightarrow 3 \times 3$ output

- Replicate this column of hidden neurons across space, with some **stride**.

Convolution Layers

0	0	0	0	0				
0								
0								
0								
0								

Example:

7×7 input

assume 3×3 connectivity

stride 1

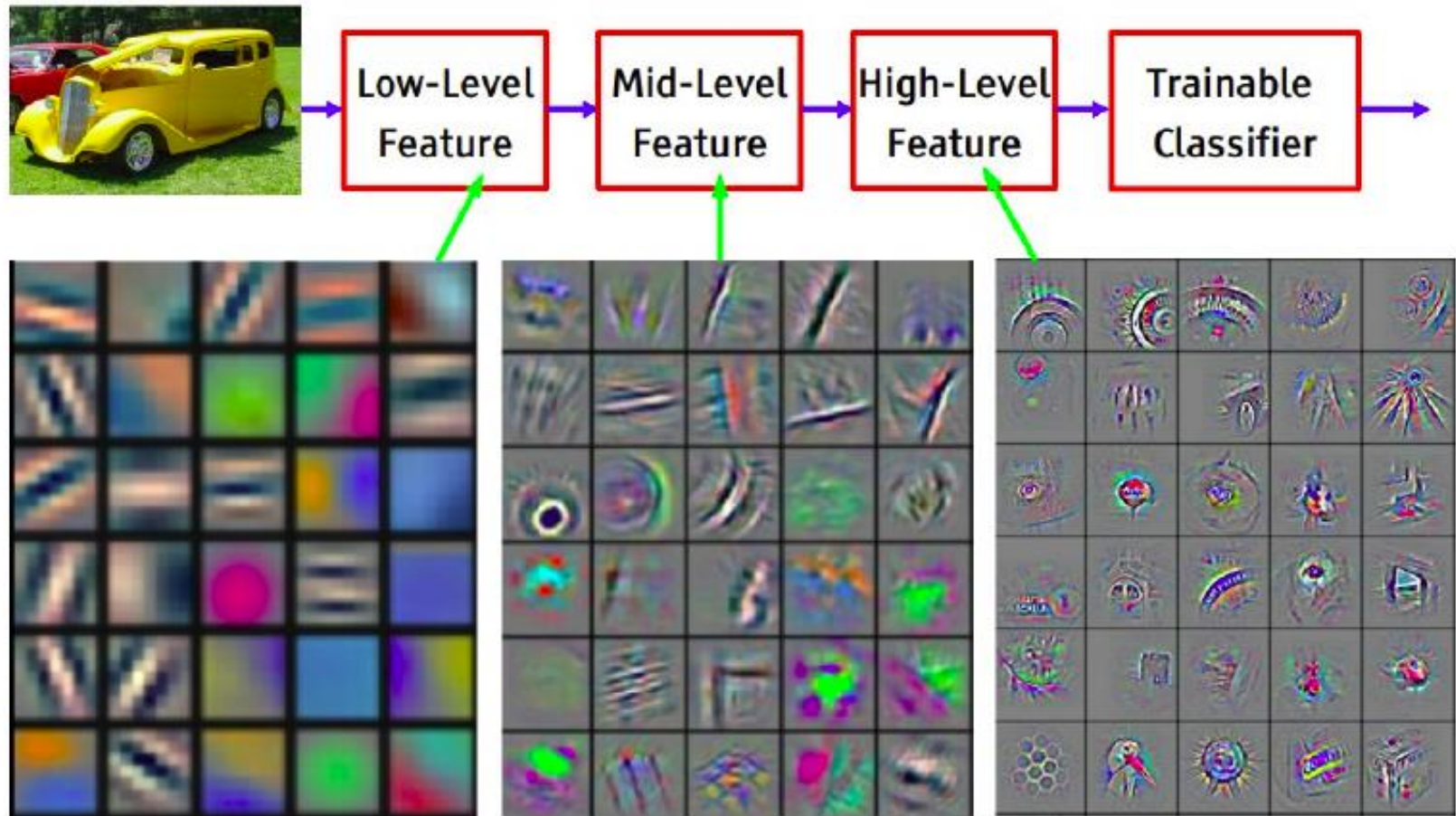
$\Rightarrow 5 \times 5$ output

What about stride 2?

$\Rightarrow 3 \times 3$ output

- Replicate this column of hidden neurons across space, with some **stride**.
- In practice, common to zero-pad the border.
 - Preserves the size of the input spatially.

Effect of Multiple Convolution Layers



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

Commonly Used Nonlinearities

- Sigmoid

$$\begin{aligned} g(a) &= \sigma(a) \\ &= \frac{1}{1 + \exp\{-a\}} \end{aligned}$$

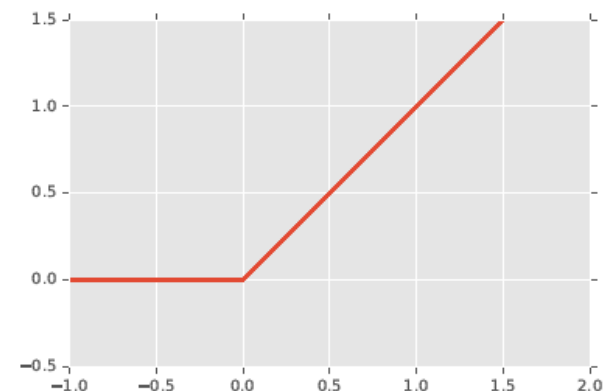
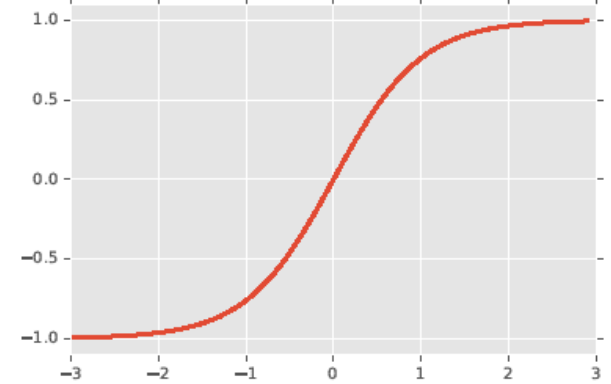
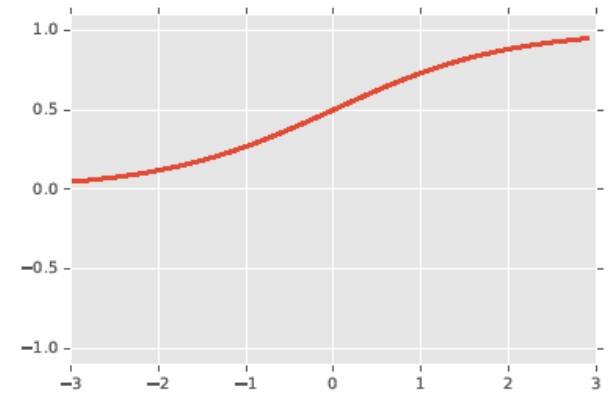
- Hyperbolic tangent

$$\begin{aligned} g(a) &= \tanh(a) \\ &= 2\sigma(2a) - 1 \end{aligned}$$

- Rectified linear unit (ReLU)

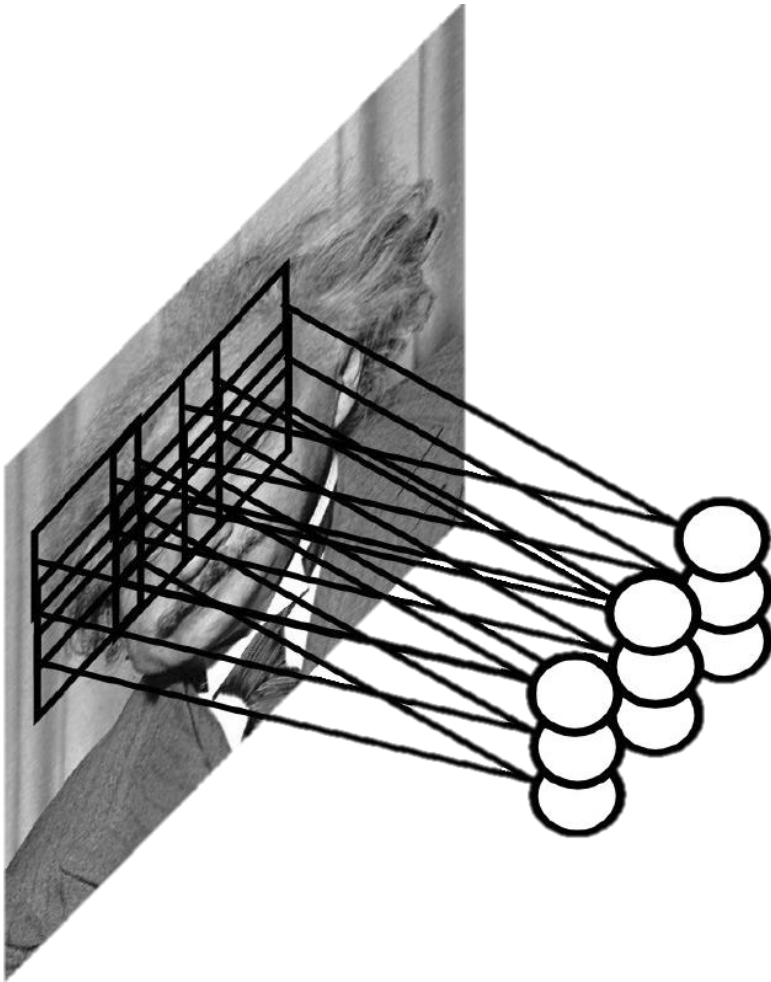
$$g(a) = \max\{0, a\}$$

Preferred option for deep networks



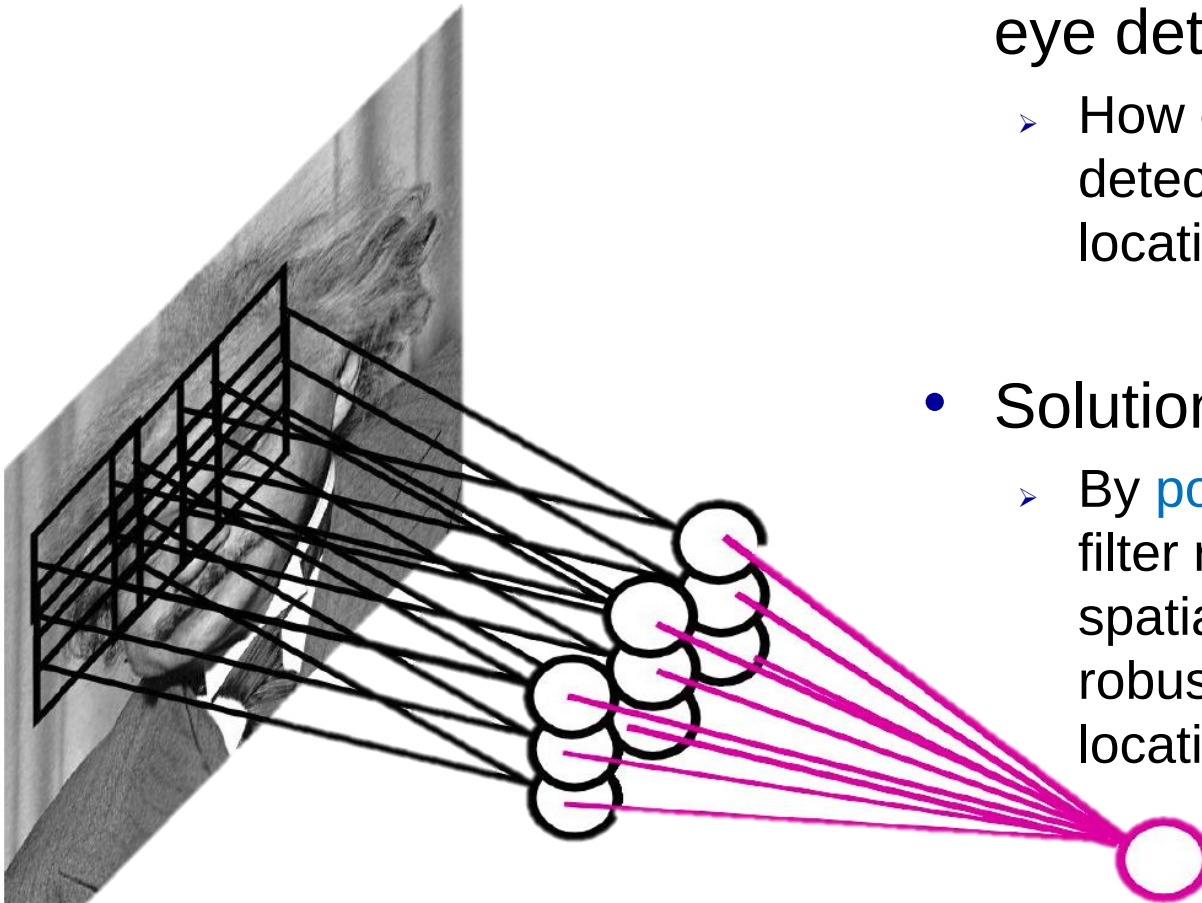
Convolutional Networks: Intuition

- Let's assume the filter is an eye detector
 - How can we make the detection robust to the exact location of the eye?

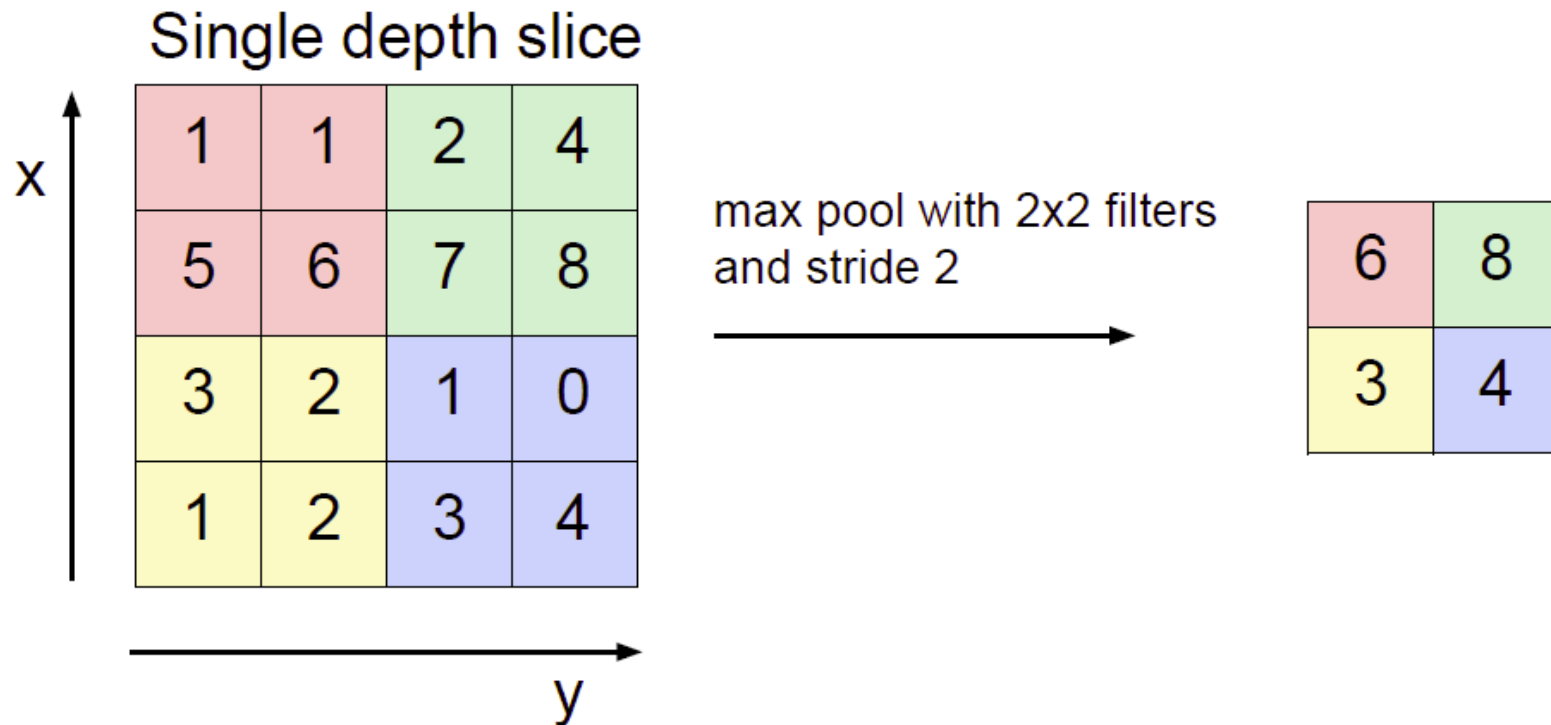


Convolutional Networks: Intuition

- Let's assume the filter is an eye detector
 - How can we make the detection robust to the exact location of the eye?
- Solution:
 - By **pooling** (e.g., max or avg) filter responses at different spatial locations, we gain robustness to the exact spatial location of features.

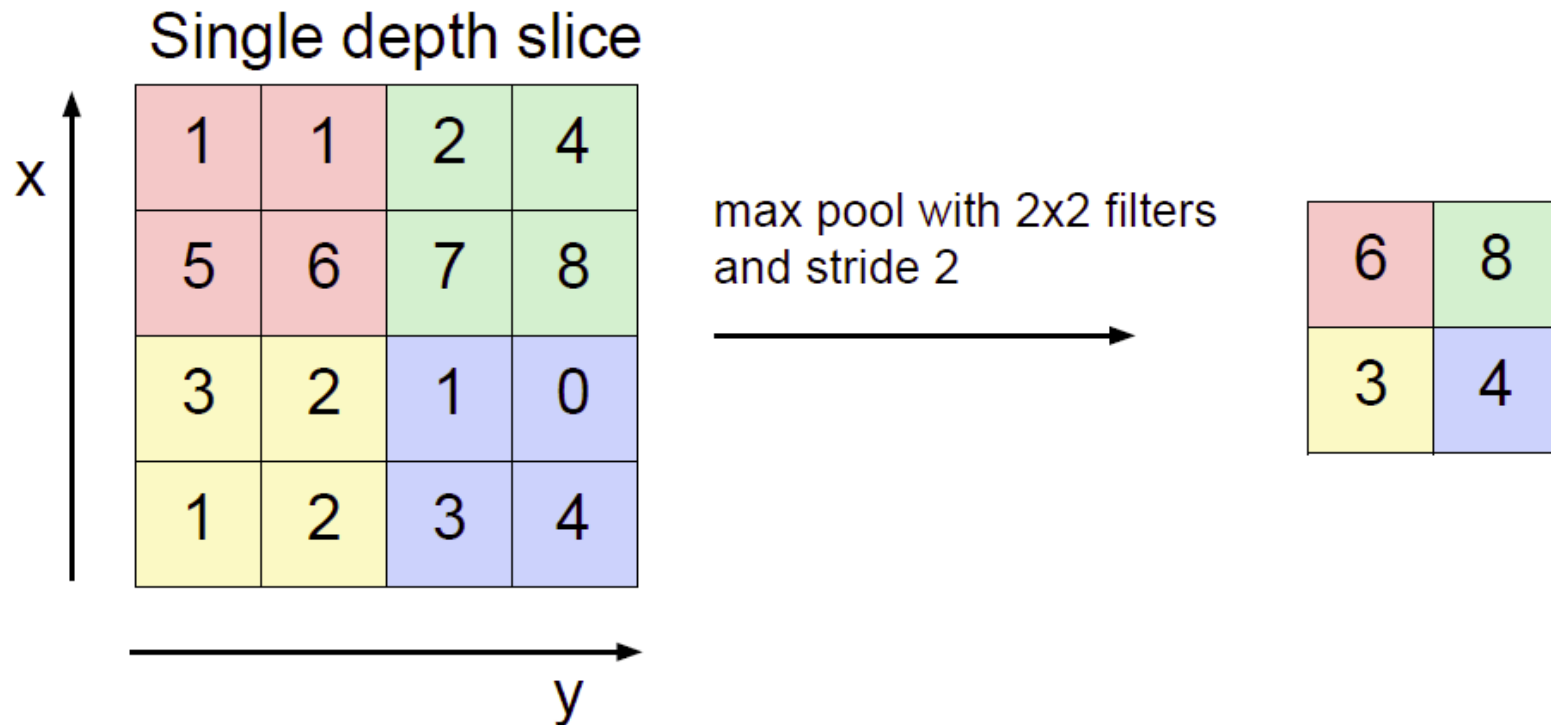


Max Pooling



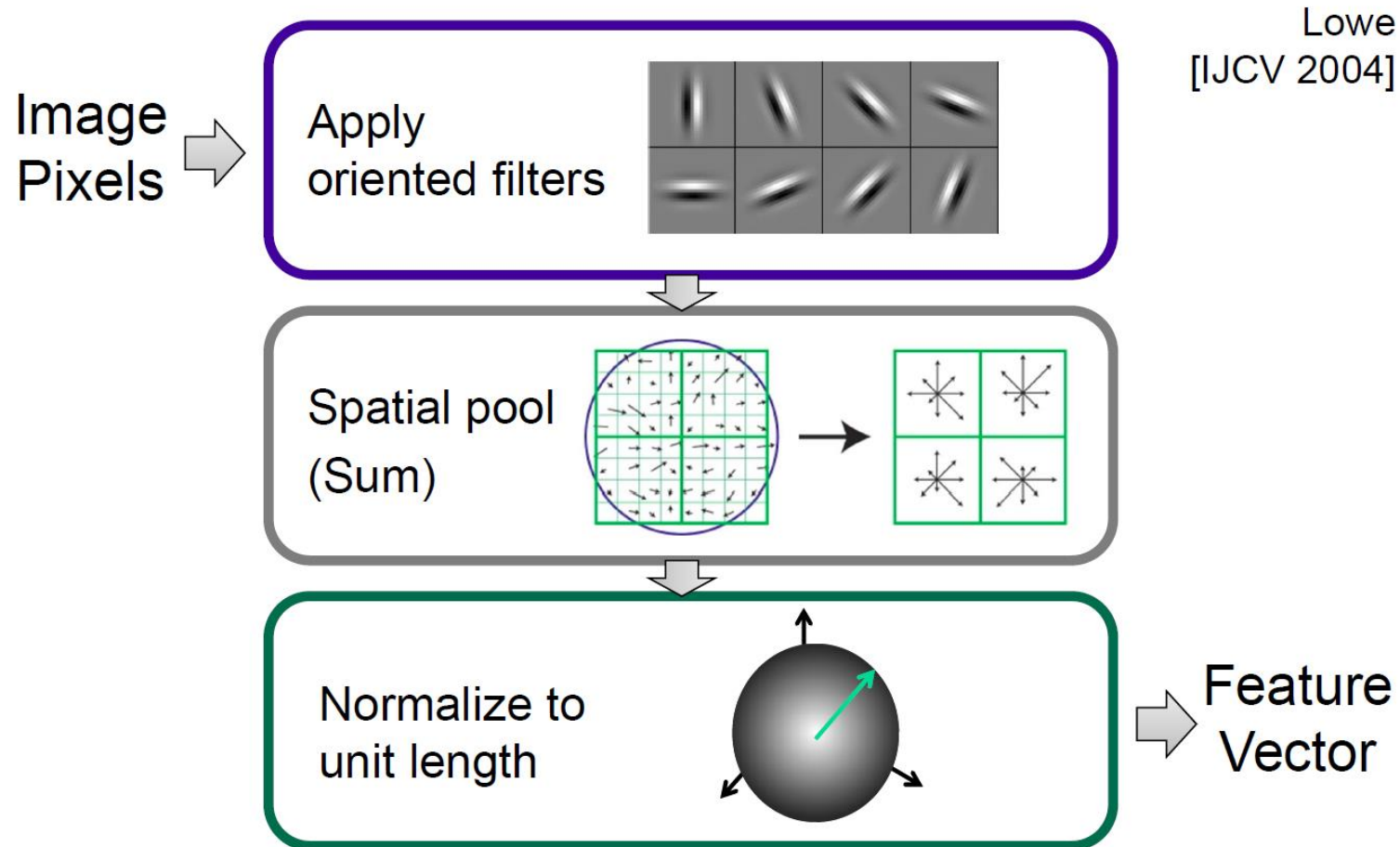
- Effect:
 - Make the representation smaller without losing too much information
 - Achieve robustness to translations

Max Pooling



- Note
 - Pooling happens independently across each slice, preserving the number of slices.

Compare: SIFT Descriptor



Compare: Spatial Pyramid Matching

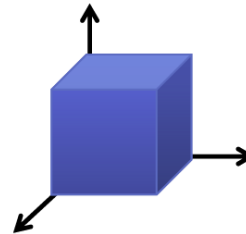
SIFT
features →

Filter with
Visual Words

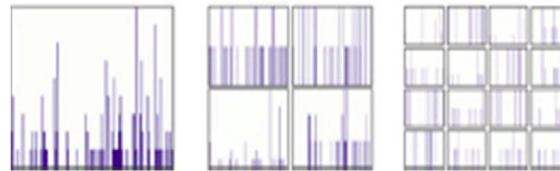


Lazebnik,
Schmid,
Ponce
[CVPR 2006]

Take max VW
response (L-inf
normalization)



Multi-scale
spatial pool
(Sum)



→ Global
image
descriptor

References and Further Reading

- More information on Deep Learning and CNNs can be found in Chapters 6 and 9 of the Goodfellow & Bengio book

I. Goodfellow, Y. Bengio, A. Courville
Deep Learning
MIT Press, 2016
<http://www.deeplearningbook.org/>

